

第 14 章 RCC—使用 HSE/HSI 配置时钟

本章参考资料：《STM32F4xx 中文参考手册》RCC 章节。

学习本章时，配合《STM32F4xx 中文参考手册》RCC 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

RCC：reset clock control 复位和时钟控制器。本章我们主要讲解时钟部分，特别是要着重理解时钟树，理解了时钟树，F407 的一切时钟的来龙去脉都会了如指掌。

14.1 RCC 主要作用—时钟部分

设置系统时钟 SYSCLK、设置 AHB 分频因子（决定 HCLK 等于多少）、设置 APB2 分频因子（决定 PCLK2 等于多少）、设置 APB1 分频因子（决定 PCLK1 等于多少）、设置各个外设的分频因子；控制 AHB、APB2 和 APB1 这三条总线时钟的开启、控制每个外设的时钟的开启。对于 SYSCLK、HCLK、PCLK2、PCLK1 这四个时钟的配置一般是： $HCLK = SYSCLK = PLLCLK$ ， $PCLK1 = HCLK/2$ ， $PCLK2 = HCLK/4$ 。

这个时钟配置也是库函数的标准配置，我们用的最多的就是这个。

14.2 RCC 框图剖析—时钟树

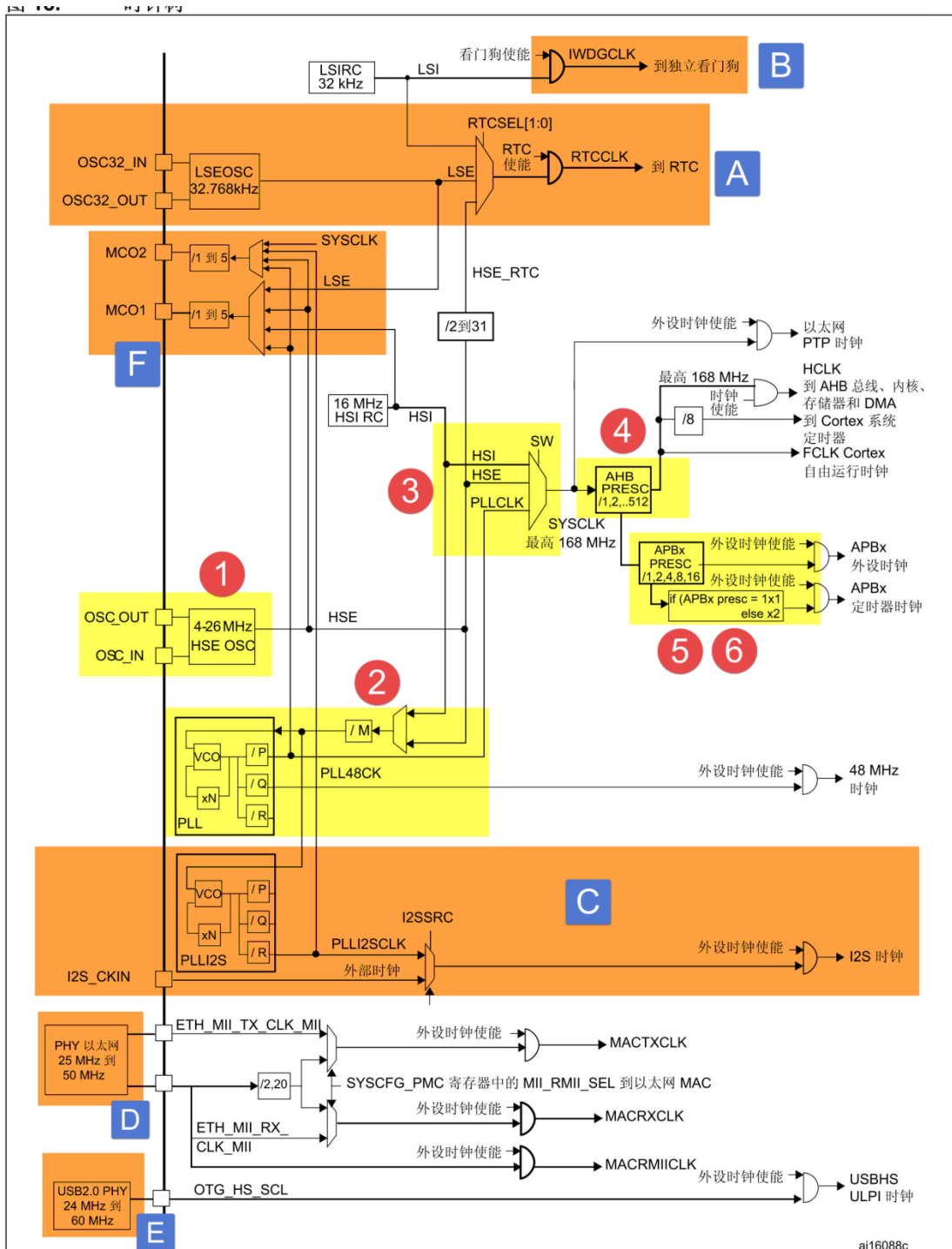
时钟树单纯讲理论的话会比较枯燥，如果选取一条主线，并辅以代码，先主后次讲解的话会很容易，而且记忆还更深刻。我们这里选取库函数时钟系统时钟函数：SetSysClock(); 以这个函数的编写流程来讲解时钟树，这个函数也是我们用库的时候默认的系统时钟设置函数。该函数的功能是利用 HSE 把时钟设置为：

$HCLK = SYSCLK = PLLCLK = 168M$ (F407) $180M$ (F429),

$PCLK1 = HCLK/2 = 84M$ (F407) $90M$ (F429),

$PCLK1 = HCLK / 4 = 42M$ (F407) $45M$ (F429)

下面我们就以这个代码的流程为主线，来分析时钟树，对应的是图中的黄色部分，代码流程在时钟树中以数字的大小顺序标识。



14.2.1 系统时钟

14.2.1.1 HSE 高速外部时钟信号

HSE 是高速的外部时钟信号，可以由有源晶振或者无源晶振提供，频率从 4-26MHz 不等。当使用有源晶振时，时钟从 OSC_IN 引脚进入，OSC_OUT 引脚悬空，当选用无源晶振时，时钟从 OSC_IN 和 OSC_OUT 进入，并且要配谐振电容。HSE 我们使用 25M 的无源晶振。如果我们使用 HSE 或者 HSE 经过 PLL 倍频之后的时钟作为系统时钟 SYSCLK，当 HSE 故障时候，不仅 HSE 会被关闭，PLL 也会被关闭，此时高速的内部时钟信号 HSI 会作为备用的系统时钟，直到 HSE 恢复正常，HSI=16M。

14.2.1.2 锁相环 PLL

PLL 的主要作用是对时钟进行倍频，然后把时钟输出到各个功能部件。PLL 有两个，一个是主 PLL，另外一个专用的 PLLI2S，他们均由 HSE 或者 HSI 提供时钟输入信号。

主 PLL 有两路的时钟输出，第一个输出时钟 PLLCLK 用于系统时钟，F407 里面最高是 168M，F429 里面最高是 180M，第二个输出用于 USB OTG FS 的时钟（48M）、RNG 和 SDIO 时钟（≤48M）。专用的 PLLI2S 用于生成精确时钟，给 I2S 提供时钟。

HSE 或者 HSI 经过 PLL 时钟输入分频因子 M（2~63）分频后，成为 VCO 的时钟输入，VCO 的时钟必须在 1~2M 之间，我们选择 HSE=25M 作为 PLL 的时钟输入，M 设置为 25，那么 VCO 输入时钟就等于 1M。

F407 的情况：

VCO 输入时钟经过 VCO 倍频因子 N 倍频之后，成为 VCO 时钟输出，VCO 时钟必须在 192~432M 之间。我们配置 N 为 336，则 VCO 的输出时钟等于 336M。如果要把系统时钟超频，就得在 VCO 倍频系数 N 这里做手脚。 $PLLCLK_OUTMAX = VCOCLK_OUTMAX / P_MIN = 432 / 2 = 216M$ ，即 F407 最高可超频到 216M。

VCO 输出时钟之后有三个分频因子：PLLCLK 分频因子 p，USB OTG FS/RNG/SDIO 时钟分频因子 Q，分频因子 R（F446 才有，F407 没有）。p 可以取值 2、4、6、8，我们配置为 2，则得到 PLLCLK=168M。Q 可以取值 4~15，但是 USB OTG FS 必须使用 48M，F407 的 Q=VCO 输出时钟

336/48=7，有关 PLL 的配置有一个专门的 RCC PLL 配置寄存器 RCC_PLLCFGR，具体描述看手册即可。

PLL 的时钟配置经过，稍微整理下可由如下公式表达：

$$\text{VCOCLK_IN} = \text{PLLCLK_IN} / M = \text{HSE} / 25 = 1\text{M}$$

$$\text{VCOCLK_OUT} = \text{VCOCLK_IN} * N = 1\text{M} * 336 = 336\text{M}$$

$$\text{PLLCLK_OUT} = \text{VCOCLK_OUT} / P = 336 / 2 = 168\text{M} \text{ (F407)} \quad 180\text{M} \text{ (F429)}$$

$$\text{USBCLK} = \text{VCOCLK_OUT} / Q = 336 / 7 = 48。$$

F429 的情况：

我们配置 N 为 360，则 VCO 的输出时钟等于 360M。如果要把系统时钟超频，就得在 VCO 倍频系数 N 这里做手脚。PLLCLK_OUTMAX = VCOCLK_OUTMAX/P_MIN = 432/2 = 216M，即 F429 最高可超频到 216M。细心的读者应该发现了，在使用 USB 的时候，PLLCLK 被降低到了 168M，不能使用 180M，这实乃 ST 的一个奇葩设计。有关 PLL 的配置有一个专门的 RCC PLL 配置寄存器 RCC_PLLCFGR，具体描述看手册即可。

VCO 输出时钟之后有三个分频因子：PLLCLK 分频因子 p，USB OTG FS/RNG/SDIO 时钟分频因子 Q，分频因子 R (F446 才有，F429 没有)。p 可以取值 2、4、6、8，我们配置为 2，则得到 PLLCLK=180M。Q 可以取值 4~15，但是 USB OTG FS 必须使用 48M，于是 Q=VCO 输出时钟 360/48=7.5，出现了小数这明显是错误，权衡之策是重新配置 VCO 的倍频因子 N=336，VCOCLK=1M*336=336M，PLLCLK=VCOCLK/2=168M，USBCLK=336/7=48M，PLL 的时钟配置经过，稍微整理下可由如下公式表达：

$$\text{VCOCLK_IN} = \text{PLLCLK_IN} / M = \text{HSE} / 25 = 1\text{M}$$

$$\text{VCOCLK_OUT} = \text{VCOCLK_IN} * N = 1\text{M} * 360 = 360\text{M}$$

$$\text{PLLCLK_OUT} = \text{VCOCLK_OUT} / P = 360 / 2 = 180\text{M}$$

$$\text{USBCLK} = \text{VCOCLK_OUT} / Q = 360 / 7 = 51.7。$$

暂时这样配置，到真正使用 USB 的时候会重新配置。

14.2.1.3 系统时钟 SYSCLK

系统时钟来源可以是：HSI、PLLCLK、HSE，具体的由时钟配置寄存器 RCC_CFGR 的 SW 位配置。我们这里设置系统时钟：SYSCLK = PLLCLK = 168M (F407) 180M (F429)。如果系统时钟是由 HSE 经过 PLL 倍频之后的 PLLCLK 得到，当 HSE 出现故障的时候，系统时钟会切换为 HSI=16M，直到 HSE 恢复正常为止。

14.2.1.4 AHB 总线时钟 HCLK

系统时钟 SYSCLK 经过 AHB 预分频器分频之后得到时钟叫 AHB 总线时钟，即 HCLK，分频因子可以是：[1,2,4, 8, 16, 64, 128, 256, 512]，具体的由时钟配置寄存器 RCC_CFGR 的 HPRE 位设置。片上大部分外设的时钟都是经过 HCLK 分频得到，至于 AHB 总线上的外设的时钟设置为多少，得等到我们使用该外设的时候才设置，我们这里只需粗线条的设置好 APB 的时钟即可。我们这里设置为 1 分频，即 HCLK=SYSCLK=168M (F407) 180M (F429)。

14.2.1.5 APB2 总线时钟 PCLK2

APB2 总线时钟 PCLK2 由 HCLK 经过高速 APB2 预分频器得到，分频因子可以是：[1,2,4, 8, 16]，具体由时钟配置寄存器 RCC_CFGR 的 PPRE2 位设置。PCLK2 属于高速的总线时钟，片上高速的外设就挂载到这条总线上，比如全部的 GPIO、USART1、SPI1 等。至于 APB2 总线上的外设的时钟设置为多少，得等到我们使用该外设的时候才设置，我们这里只需粗线条的设置好 APB2 的时钟即可。我们这里设置为 2 分频，即 PCLK2 = HCLK / 2 = 84M (F407) 90M (F429)。

14.2.1.6 APB1 总线时钟 PCLK1

APB1 总线时钟 PCLK1 由 HCLK 经过低速 APB 预分频器得到，分频因子可以是：[1,2,4, 8, 16]，具体由时钟配置寄存器 RCC_CFGR 的 PPRE1 位设置。PCLK1 属于低速的总线时钟，最高为 42M (F407) 45M (F429)，片上低速的外设就挂载到这条总线上，比如 USART2/3/4/5、SPI2/3、I2C1/2 等。至于 APB1 总线上的外设的时钟设置为多少，得等到我们使用该外设的时候才设置，我们这里只需粗线条的设置好 APB1 的时钟即可。我们这里设置为 4 分频，即 PCLK1 = HCLK / 4 = 42M (F407) 45M (F429)。

14.2.1.7 设置系统时钟库函数

上面的 6 个步骤对应的设置系统时钟库函数具体见代码清单 15 1，为了方便阅读，已经把跟 407 不相关的代码删掉，把英文注释翻译成了中文，并把代码标上了序号，总共 6 个步骤。该函数是直接操作寄存器的，有关寄存器部分请参考数据手册的 RCC 的寄存器描述部分。

列表 1: 代码 15-1 设置系统时钟库函数

```
/**
 * @brief  System Clock Configuration
 *
 *      The system Clock is configured as follow :
 *
 *      System Clock source            = PLL (HSE)
 *      SYSCLK(Hz)                     = 168000000
 *      HCLK(Hz)                      = 168000000
 *      AHB Prescaler                  = 1
 *      APB1 Prescaler                 = 4
 *      APB2 Prescaler                 = 2
 *      HSE Frequency(Hz)              = 8000000
 *      PLL_M                          = 25
 *      PLL_N                          = 336
 *      PLL_P                          = 2
 *      PLL_Q                          = 7
 *      VDD(V)                         = 3.3
 *      Main regulator output voltage  = Scale1 mode
 *      Flash Latency(WS)              = 5
 *
 * @param  None
 * @retval None
 */
static void SystemClock_Config(void)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct;
    RCC_OscInitTypeDef RCC_OscInitStruct;
```

(下页继续)

(续上页)

```
/* Enable Power Control clock */
__HAL_RCC_PWR_CLK_ENABLE();

/* The voltage scaling allows optimizing the power consumption
when the device is clocked below the maximum system frequency,
to update the voltage scaling value
   regarding system frequency refer to product datasheet. */
__HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

/* Enable HSE Oscillator and activate PLL with HSE as source */
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState = RCC_HSE_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLM = 25;
RCC_OscInitStruct.PLL.PLLN = 336;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
RCC_OscInitStruct.PLL.PLLQ = 7;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
    while (1) {};
}

/* Select PLL as system clock source and configure the HCLK,
PCLK1 and PCLK2 clocks dividers */
RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK |
RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2);
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) != HAL_OK)
    ↪{
```

(下页继续)

(续上页)

```
        while (1) {};\n    }\n\n    /* STM32F405x/407x/415x/417x Revision Z devices:prefetch is supported ↵\n    ↵ */\n    if (HAL_GetREVID() == 0x1001) {\n        /* Enable the Flash prefetch */\n        __HAL_FLASH_PREFETCH_BUFFER_ENABLE();\n    }\n}\n}
```

14.2.2 其他时钟

通过对系统时钟设置的讲解，整个时钟树我们已经把握的有六七成，剩下的时钟部分我们讲解几个重要的。

14.2.2.1 A、RTC 时钟

RTCCLK 时钟源可以是 HSE 1 MHz (HSE 由一个可编程的预分频器分频)、LSE 或者 LSI 时钟。选择方式是编程 RCC 备份域控制寄存器 (RCC_BDCR) 中的 RTCSEL[1:0] 位和 RCC 时钟配置寄存器 (RCC_CFGR) 中的 RTCPRE[4:0] 位。所做的选择只能通过复位备份域的方式修改。我们通常的做法是由 LSE 给 RTC 提供时钟，大小为 32.768KHZ。LSE 由外接的晶体谐振器产生，所配的谐振电容精度要求高，不然很容易不起震。

14.2.2.2 B、独立看门狗时钟

独立看门狗时钟由内部的低速时钟 LSI 提供，大小为 32KHZ。

14.2.2.3 C、I2S 时钟

I2S 时钟可由外部的时钟引脚 I2S_CKIN 输入，也可由专用的 PLLI2SCLK 提供，具体的由 RCC 时钟配置寄存器 (RCC_CFGR) 的 I2SSCR 位配置。我们在使用 I2S 外设驱动 W8978 的时候，使用的时钟是 PLLI2SCLK，这样就可以省掉一个有源晶振。

14.2.2.4 D、PHY 以太网时钟

F407 要想实现以太网功能，除了有本身内置的 MAC 之外，还需要外接一个 PHY 芯片，常见的 PHY 芯片有 DP83848 和 LAN8720，其中 DP83848 支持 MII 和 RMII 接口，LAN8720 只支持 RMII 接口。野火 F407 开发板用的是 RMII 接口，选择的 PHY 芯片是 LAN8720。使用 RMII 接口的好处是使用的 IO 减少了一半，速度还是跟 MII 接口一样。当使用 RMII 接口时，PHY 芯片只需输出一路时钟给 MCU 即可，如果是 MII 接口，PHY 芯片则需要提供两路时钟给 MCU。

14.2.2.5 E、USB PHY 时钟

F407 的 USB 没有集成 PHY，要想实现 USB 高速传输的话，必须外置 USB PHY 芯片，常用的芯片是 USB3300。当外接 USB PHY 芯片时，PHY 芯片需要给 MCU 提供一个时钟。

外扩 USB3300 会占用非常多的 IO，跟 SDRAM 和 RGB888 的 IO 会复用的很厉害，鉴于 USB 高速传输用的比较少，野火 F407 霸天虎就没有外扩这个芯片。

14.2.2.6 F、MCO 时钟输出

MCO 是 microcontroller clock output 的缩写，是微控制器时钟输出引脚，主要作用是可以对外提供时钟，相当于一个有源晶振。F407 中有两个 MCO，由 PA8/PC9 复用所得。MCO1 所需的时钟源通过 RCC 时钟配置寄存器 (RCC_CFGR) 中的 MCO1PRE[2:0] 和 MCO1[1:0] 位选择。MCO2 所需的时钟源通过 RCC 时钟配置寄存器 (RCC_CFGR) 中的 MCO2PRE[2:0] 和 MCO2 位选择。有关 MCO 的 IO、时钟选择和输出速率的具体信息如下表所示：

时钟输出	IO	时钟来源	最大输出速率
MCO1	PA8	HSI、LSE、HSE、PLLCLK	100M
MCO2	PC9	HSE、PLLCLK、SYSCLK、PLLI2SCLK	100M

14.3 配置系统时钟实验

注意：此处为 F407 霸天虎程序介绍，F429 挑战者的原理也是一致的。

14.3.1 使用 HSE

一般情况下，我们都是使用 HSE，然后 HSE 经过 PLL 倍频之后作为系统时钟。F407 系统时钟最高为 168M，这个是官方推荐的最高的稳定时钟，如果你想铤而走险，也可以超频，超频最高能到 216M。

如果我们使用库函数编程，当程序来到 main 函数之前，启动文件：startup_stm32f40xxx.s 已经调用 SystemInit() 函数把系统时钟初始化成 168MHZ，SystemInit() 在库文件：system_stm32f4xx.c 中定义。如果我们想把系统时钟设置低一点或者超频的话，可以修改底层的库文件，但是为了维持库的完整性，我们可以根据时钟树的流程自行写一个。

14.3.2 使用 HSI

当 HSE 直接或者间接（HSE 经过 PLL 倍频）的作为系统时钟的时候，如果 HSE 发生故障，不仅 HSE 会被关闭，连 PLL 也会被关闭，这个时候系统会自动切换 HSI 作为系统时钟，此时 SYSCLK=HSI=16M，如果没有开启 CSS 和 CSS 中断的话，那么整个系统就只能在低速率运行，这是系统跟瘫痪没什么两样。

如果开启了 CSS 功能的话，那么可以当 HSE 故障时，在 CSS 中断里面采取补救措施，使用 HSI，重新设置系统频率为 168M，让系统恢复正常使用。但这只是权宜之计，并非万全之策，最好的

方法还是要采取相应的补救措施并报警，然后修复 HSE。临时使用 HSI 只是为了把损失降低到最小，毕竟 HSI 较于 HSE 精度还是要低点。

F103 系列中，使用 HSI 最大只能把系统设置为 64M，并不能跟使用 HSE 一样把系统时钟设置为 72M，究其原因是 HSI 在进入 PLL 倍频的时候必须 2 分频，导致 PLL 倍频因子调到最大也只能到 64M，而 HSE 进入 PLL 倍频的时候则不用 2 分频。

在 F407 中，无论是使用 HSI 还是 HSE 都可以把系统时钟设置为 168M，因为 HSE 或者 HSI 在进入 PLL 倍频的时候都会被分频为 1M 之后再倍频。

还有一种情况是，有些用户不想用 HSE，想用 HSI，但是又不知道怎么用 HSI 来设置系统时钟，因为调用库函数都是使用 HSE，下面我们给出个使用 HSI 配置系统时钟例子，起个抛砖引玉的作用。

14.3.3 硬件设计

1. RCC
2. LED 一个

RCC 是单片机内部资源，不需要外部电路。通过 LED 闪烁的频率来直观的判断不同系统时钟频率对软件延时的效果。

14.3.4 软件设计

我们编写两个 RCC 驱动文件，bsp_clkconfig.h 和 bsp_clkconfig.c，用来存放 RCC 系统时钟配置函数。

14.3.4.1 编程要点

编程要点对应着时钟树图中的序号。

- 1、开启 HSE/HSI，并等待 HSE/HSI 稳定
- 2、设置 AHB、APB2、APB1 的预分频因子

- 3、设置 PLL 的时钟来源，设置 VCO 输入时钟分频因子 PLL_M，设置 VCO 输出时钟倍频因子 PLL_N，设置 PLLCLK 时钟分频因子 PLL_P，设置 OTG FS,SDIO,RNG 时钟分频因子 PLL_Q。
- 4、开启 PLL，并等待 PLL 稳定
- 5、把 PLLCK 切换为系统时钟 SYSCLK
- 6、读取时钟切换状态位，确保 PLLCLK 被选为系统时钟

14.3.4.2 代码分析

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。

14.3.4.2.1 使用 HSE 配置系统时钟

列表 2: 代码 15-2 HSE 作为系统时钟来源

```
/*
 * 使用 HSE 时，设置系统时钟的步骤
 * 1、开启 HSE，并等待 HSE 稳定
 * 2、设置 AHB、APB2、APB1 的预分频因子
 * 3、设置 PLL 的时钟来源
 *     设置 VCO 输入时钟 分频因子          m
 *     设置 VCO 输出时钟 倍频因子          n
 *     设置 PLLCLK 时钟分频因子            p
 *     设置 OTG FS,SDIO,RNG 时钟分频因子 q
 * 4、开启 PLL，并等待 PLL 稳定
 * 5、把 PLLCK 切换为系统时钟 SYSCLK
 * 6、读取时钟切换状态位，确保 PLLCLK 被选为系统时钟
 */
/*
 * m: VCO 输入时钟 分频因子，取值 2~63
```

(下页继续)

(续上页)

```
* n: VCO 输出时钟 倍频因子, 取值 50~432
* p: PLLCLK 时钟分频因子 , 取值 2, 4, 6, 8
* q: OTG FS, SDIO, RNG 时钟分频因子, 取值 4~15
* 函数调用举例, 使用 HSE 设置时钟
* SYSCLK=HCLK=168MHz, PCLK2=HCLK/2=84MHz, PCLK1=HCLK/4=42MHz
* HSE_SetSysClock(25, 336, 2, 7);
*
HSE 作为时钟来源, 经过 PLL 倍频作为系统时钟, 这是通常的做法

*/
void HSE_SetSysClock(uint32_t m, uint32_t n, uint32_t p, uint32_t q)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct;
    RCC_OscInitTypeDef RCC_OscInitStruct;

    /* Enable Power Control clock */
    __HAL_RCC_PWR_CLK_ENABLE();

    /* The voltage scaling allows optimizing the power consumption when
    the device is clocked below the maximum system frequency, to update
    the voltage scaling value regarding system frequency refer to product
    datasheet. */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /* Enable HSE Oscillator and activate PLL with HSE as source */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = m;
    RCC_OscInitStruct.PLL.PLLN = n;
    RCC_OscInitStruct.PLL.PLLP = p;
```

(下页继续)

(续上页)

```
RCC_OscInitStruct.PLL.PLLQ = q;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
    while (1) {};
}

/* Select PLL as system clock source and configure the HCLK,
PCLK1 and PCLK2 clocks dividers */
RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK |
                                RCC_CLOCKTYPE_HCLK |
                                RCC_CLOCKTYPE_PCLK1 |
                                RCC_CLOCKTYPE_PCLK2);

RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) !=
    HAL_OK) {
    while (1) {};
}

/* STM32F405x/407x/415x/417x Revision Z devices: prefetch is supported
↪ */
if (HAL_GetREVID() == 0x1001) {
    /* Enable the Flash prefetch */
    __HAL_FLASH_PREFETCH_BUFFER_ENABLE();
}
}
```

这个函数采用库函数编写，代码理解参考注释即可。函数有 4 个形参 m、n、p、q，具体说明见表 15-1 PLL 配置因子。

表 15-1 PLL 配置因子

形参	形参说明	取值范围
m	VCO 输入时钟分频因子	2~63
n	VCO 输出时钟倍频因子	192~432
p	PLLCLK 时钟分频因子	2/4/6/8
q	OTGFS,SDIO,RNG 时钟分频因子	4~15

HSE 我们使用 25M，参数 m 我们一般也设置为 25，所以我们需要修改系统时钟的时候只需要修改参数 n 和 p 即可， $SYSCLK=PLLCLK=HSE/m*n/p$ 。

函数调用举例：HSE_SetSysClock(25, 336, 2, 7) 把系统时钟设置为 168M，这个跟库里面的系统时钟配置是一样的。HSE_SetSysClock(25, 432, 2, 9) 把系统时钟设置为 216M，这个是超频，要慎用。

14.3.4.2.2 使用 HSI 配置系统时钟

```
/*
 * 使用 HSI 时，设置系统时钟的步骤
 * 1、开启 HSI，并等待 HSI 稳定
 * 2、设置 AHB、APB2、APB1 的预分频因子
 * 3、设置 PLL 的时钟来源
 *    设置 VCO 输入时钟 分频因子      m
 *    设置 VCO 输出时钟 倍频因子      n
 *    设置 SYSCLK 时钟分频因子        p
 *    设置 OTG FS,SDIO,RNG 时钟分频因子 q
 * 4、开启 PLL，并等待 PLL 稳定
 * 5、把 PLLCK 切换为系统时钟 SYSCLK
 * 6、读取时钟切换状态位，确保 PLLCLK 被选为系统时钟
 */

/*
 * m: VCO 输入时钟 分频因子，取值 2~63
 * n: VCO 输出时钟 倍频因子，取值 192~432
```

(下页继续)

(续上页)

```
* p: PLLCLK 时钟分频因子 , 取值 2, 4, 6, 8
* q: OTG FS, SDIO, RNG 时钟分频因子, 取值 4~15
* 函数调用举例, 使用 HSI 设置时钟
* SYSCLK=HCLK=168M, PCLK2=HCLK/2=84M, PCLK1=HCLK/4=42M
* HSI_SetSysClock(16, 336, 2, 7);
* HSE 作为时钟来源, 经过 PLL 倍频作为系统时钟, 这是通常的做法
void HSI_SetSysClock(uint32_t m, uint32_t n, uint32_t p, uint32_t q)
{
    RCC_ClkInitTypeDef RCC_ClkInitStruct;
    RCC_OscInitTypeDef RCC_OscInitStruct;

    /* Enable Power Control clock */
    __HAL_RCC_PWR_CLK_ENABLE();

    /* The voltage scaling allows optimizing the power consumption
    when the device is clocked below the maximum system frequency,
    to update the voltage scaling value regarding system
    frequency refer to product datasheet. */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /* Enable HSE Oscillator and activate PLL with HSE as source */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = m;
    RCC_OscInitStruct.PLL.PLLN = n;
    RCC_OscInitStruct.PLL.PLLP = p;
    RCC_OscInitStruct.PLL.PLLQ = q;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
        while (1) {};
    }
}
```

(下页继续)

(续上页)

```
/* Select PLL as system clock source and configure the HCLK,
PCLK1 and PCLK2 clocks dividers */
RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK |
                                RCC_CLOCKTYPE_HCLK |
                                RCC_CLOCKTYPE_PCLK1 |
                                RCC_CLOCKTYPE_PCLK2);
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) !=
    HAL_OK) {
    while (1) {};
}

/* STM32F405x/407x/415x/417x Revision Z devices: prefetch is supported ↵
↵ */
if (HAL_GetREVID() == 0x1001) {
    /* Enable the Flash prefetch */
    __HAL_FLASH_PREFETCH_BUFFER_ENABLE();
}
}
```

这个函数采用库函数编写，代码理解参考注释即可。函数有 4 个形参 m、n、p、q，具体说明见表 15-2 PLL 配置因子。

表 15-2 PLL 配置因子

形参	形参说明	取值范围
m	VCO 输入时钟分频因子	2~63
n	VCO 输出时钟倍频因子	192~432
p	PLLCLK 时钟分频因子	2/4/6/8
q	OTGFS,SDIO,RNG 时钟分频因子	4~15

HSI 为 16M，参数 m 我们一般也设置为 16，所以我们需要修改系统时钟的时候只需要修改参数 n 和 p 即可， $SYSCLK=PLLCLK=HSI/m*n/p$ 。

函数调用举例：HSI_SetSysClock(16, 336, 2, 7) 把系统时钟设置为 168M，这个跟库里面的系统时钟配置是一样的。HSI_SetSysClock(16, 432, 2, 9) 把系统时钟设置为 216M，这个是超频，要慎用。

14.3.4.2.3 软件延时

```
void Delay(__IO uint32_t nCount)
{
    for (; nCount != 0; nCount--);
}
```

软件延时函数，使用不同的系统时钟，延时时间不一样，可以通过 LED 闪烁的频率来判断。

14.3.4.2.4 MCO 输出

在 F407 中，PA8/PC9 可以复用为 MCO1/2 引脚，对外提供时钟输出，我们也可以用示波器监控该引脚的输出来判断我们的系统时钟是否设置正确野火 F407 开发板中 PA8 与 PC9 都引出来了，在摄像头接口位置，都可以使用示波器来监控波形。

14.3.4.2.5 主函数

```
// 使用 HSE 或者 HSI 配置系统时钟
#include "stm32f4xx.h"
#include "../rcc/bsp_clkconfig.h"
#include "../led/bsp_led.h"

void Delay(__IO u32 nCount);

/**
 * @brief 主函数
 * @param 无
 * @retval 无
 */
int main(void)
{
    // 程序来到 main 函数之前, 启动文件: startup_stm32f4xx.s 已经调用
    // SystemInit() 函数把系统时钟初始化成 168MHZ
    // SystemInit() 在 system_stm32f4xx.c 中定义
    // 如果用户想修改系统时钟, 可自行编写程序修改
    // 重新设置系统时钟, 这时候可以选择使用 HSE 还是 HSI

    // 使用 HSE, 配置系统时钟为 168M
    HSE_SetSysClock(25, 336, 2, 7);

    // 系统时钟超频到 216M 爽一下, 最高是 216M, 别往死里整
    // HSE_SetSysClock(25, 432, 2, 9);

    // 使用 HSI, 配置系统时钟为 168M
    // HSI_SetSysClock(16, 336, 2, 7);

    // LED 端口初始化
```

(下页继续)

(续上页)

```
LED_GPIO_Config();

// MCO GPIO 初始化
MCO1_GPIO_Config();
MCO2_GPIO_Config();

// MCO1 输出 PLLCLK
HAL_RCC_MCOConfig(RCC_MCO1, RCC_MCO1SOURCE_PLLCLK, RCC_MCODIV_2);

// MCO2 输出 SYSCLK
HAL_RCC_MCOConfig(RCC_MCO2, RCC_MCO2SOURCE_SYSCLK, RCC_MCODIV_2);

while (1) {
    LED1( ON );        // 亮
    Delay(0x0FFFFFFF);
    LED1( OFF );       // 灭
    Delay(0x0FFFFFFF);
}
}

void Delay(__IO uint32_t nCount)    //简单的延时函数
{
    for (; nCount != 0; nCount--);
}
```

在主函数中，可以调用 `HSE_SetSysClock()` 或者 `HSI_SetSysClock()` 这两个函数把系统时钟设置成各种常用的时钟，然后通过 MCO 引脚监控，或者通过 LED 闪烁的快慢体验不同的系统时钟对同一个软件延时函数的影响。

14.3.5 下载验证

把编译好的程序下载到开发板，可以看到设置不同的系统时钟时，LED 闪烁的快慢不一样。更精确的数据我们可以用示波器监控 MCO 引脚看到。

第 15 章 STM32 中断应用概览

本章参考资料《STM32F4xx 中文参考手册》第十章-中断和事件、《ARM Cortex™-M4F 技术参考手册》-4.3 章节：NVIC 和 4.4 章节：SCB—4.4.5 的 AIRCR。

STM32 中断非常强大，每个外设都可以产生中断，所以中断的讲解放在哪一个外设里面去讲都不合适，这里单独抽出一章来做一个总结性的介绍，这样在其他章节涉及到中断部分的知识我们就不用费很大的篇幅去讲解，只要示意性带过即可。

本章如无特别说明，异常就是中断，中断就是异常，请不要刻意钻牛角尖较劲。

15.1 异常类型

F407 在内核水平上搭载了一个异常响应系统，支持为数众多的系统异常和外部中断。其中系统异常有 10 个，外部中断有 82 个。除了个别异常的优先级被定死外，其它异常的优先级都是可编程的。有关具体的系统异常和外部中断可在 HAL 库文件 `stm32f4xx.h` 这个头文件查询到，在 `IRQn_Type` 这个结构体里面包含了 F4 系列全部的异常声明。

表格 16-1 F407 系统异常清单

编号	优先级	优先级类型	名称	说明	地址
	•	•	•	保留 (实际存的是 MSP 地址)	0X0000 0000
	-3	固定	Reset	复位	0X0000 0004
	-2	固定	NMI	不可屏蔽中断。RCC 时钟安全系统 (CSS) 连接到 NMI 向量	0X0000 0008
	-1	固定	HardFault	所有类型的错误	0X0000 000C
	0	可编程	MemManage	存储器管理	0X0000 0010
	1	可编程	BusFault	预取指失败, 存储器访问失败	0X0000 0014
	2	可编程	UsageFault	未定义的指令或非法状态	0X0000 0018
	•	•	•	保留	0X0000 001C-0X0000 002B
	3	可编程	SVCall	通过 SWI 指令调用的系统服务	0X0000 002C
	4	可编程	Debug Monitor	调试监控器	0X0000 0030
	•	•	•	保留	0X0000 0034
	5	可编程	PendSV	可挂起的系统服务	0X0000 0038
	6	可编程	SysTick	系统嘀嗒定时器	0X0000 003C
论坛: https://www.firebbs.cn/		238		天猫: https://yehuosm.tmall.com	

表格 16-2 F407 外部中断清单

表 16-2 F407 外部中断清单

编号	优先级	优先级类型	名称	说明	地址
0	7	可编程	-	窗口看门狗中断	0X0000 0040
1	8	可编程	PVD	连接 EXTI 线的可编程电压检测中断	0X0000 0044
2	9	可编程	TAMP_STAMP	连接 EXTI 线的入侵和时间戳中断	0X0000 0048
中间部分省略，详情请参考《STM32F4xx 中文参考手册》第十章-中断和事件-向量表部分					
79	86	可编程	CRYP	CRYP 加密全局中断	0X 0000 017C
80	87	可编程	HASH_RNG	哈希和随机数发生器全局中断	0X 0000 0180
81	88	可编程	FPU	FPU 全局中断	0X 0000 0184

15.2 NVIC 简介

在讲如何配置中断优先级之前，我们需要先了解下 NVIC。NVIC 是嵌套向量中断控制器，控制着整个芯片中断相关的功能，它跟内核紧密耦合，是内核里面的一个外设。但是各个芯片厂商在设计芯片的时候会对 Cortex-M4 内核里面的 NVIC 进行裁剪，把不需要的部分去掉，所以说 STM32 的 NVIC 是 Cortex-M4 的 NVIC 的一个子集。

15.2.1 NVIC 寄存器简介

在函数库中，NVIC 的结构体定义可谓是颇有远虑，给每个寄存器都预览了很多位，恐怕为的是日后扩展功能。不过 STM32F407 可用不了这么多，只是用了部分而已，具体使用了多少可参考《ARM Cortex™-M4F 技术参考手册》-4.3.11:NVIC 寄存器映射。

列表 1: 代码 16-1 NVIC 结构体定义，来自固件库头文件：core_cm3.h

```
typedef struct {
    __IO uint32_t ISER[8];           // 中断使能寄存器
    uint32_t RESERVED0[24];
    __IO uint32_t ICER[8];           // 中断清除寄存器
    uint32_t RSERVED1[24];
    __IO uint32_t ISPR[8];           // 中断使能悬起寄存器
    uint32_t RESERVED2[24];
    __IO uint32_t ICPR[8];           // 中断清除悬起寄存器
    uint32_t RESERVED3[24];
    __IO uint32_t IABR[8];           // 中断有效位寄存器
    uint32_t RESERVED4[56];
    __IO uint8_t IP[240];            // 中断优先级寄存器 (8Bit wide)
    uint32_t RESERVED5[644];
    __O uint32_t STIR;               // 软件触发中断寄存器
} NVIC_Type;
```

在配置中断的时候我们一般只用 ISER、ICER 和 IP 这三个寄存器，ISER 用来使能中断，ICER 用来失能中断，IP 用来设置中断优先级。

15.2.2 NVIC 中断配置固件库

函数库文件 core_cm4.h 的最后，还提供了 NVIC 的一些函数，这些函数遵循 CMSI 规则，只要是 Cortex-M4 的处理器都可以使用，具体如下：

表格 16-3 符合 CMSIS 标准的 NVIC 库函数

NVIC 库函数	描述
void NVIC_EnableIRQ(IRQn_Type IRQn)	使能中断
void NVIC_DisableIRQ(IRQn_Type IRQn)	失能中断
void NVIC_SetPendingIRQ(IRQn_Type IRQn)	设置中断悬起位
void NVIC_ClearPendingIRQ(IRQn_Type IRQn)	清除中断悬起位
uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)	获取悬起中断编号
void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)	设置中断优先级
uint32_t NVIC_GetPriority(IRQn_Type IRQn)	获取中断优先级
void NVIC_SystemReset(void)	系统复位

这些库函数我们在编程的时候用的都比较少，甚至基本都不用。在配置中断的时候我们还有更简洁的方法，请看中断编程小节

15.3 优先级的定义

15.3.1 优先级定义

在 NVIC 有一个专门的寄存器：中断优先级寄存器 NVIC_IPRx（在 F407 中，x=0…981）用来配置外部中断的优先级，IPR 宽度为 8bit，原则上每个外部中断可配置的优先级为 0~255，数值越小，优先级越高。但是绝大多数 CM4 芯片都会精简设计，以致实际上支持的优先级数减少，在 F407 中，只使用了高 4bit，如下所示：

表 16-4 F407 使用 4bit 表达优先级

bit7 bit6 bit5 bit4	bit3 bit2 bit1 bit0
用于表达优先级	未使用，读回为 0

用于表达优先级的这 4bit，又被分组成抢占优先级和子优先级。如果有多个中断同时响应，抢占优先级高的就会抢占抢占优先级低的优先得到执行，如果抢占优先级相同，就比较子优先级。如果抢占优先级和子优先级都相同的话，就比较他们的硬件中断编号，编号越小，优先级越高。

15.3.2 优先级分组

优先级的分组由内核外设 SCB 的应用程序中断及复位控制寄存器 AIRCR 的 PRIGROUP[10:8] 位决定，F103 分为了 5 组，具体如下：主优先级 = 抢占优先级

表 16-5

PRIGROUP[2:0]	中断优先级值 PRI_N[7:4]			级数	
	二进制点	主优先级位	子优先级位	主优先级	子优先级
0b 011	0b <u>xxxx</u>	[7:4]	None	16	None
0b 100	0b <u>xxx.y</u>	[7:5]	[4]	8	2
0b 101	0b <u>xx.yy</u>	[7:6]	[5:4]	4	4
0b 110	0b <u>x.yyy</u>	[7]	[6:4]	2	9
0b 111	0b <u>.yyyy</u>	None	[7:4]	None	16

设置优先级分组可调用库函数 NVIC_PriorityGroupConfig() 实现，有关 NVIC 中断相关的库函数都在库文件 misc.c 和 misc.h 中。

列表 2: 代码 16-2 中断优先级分组库函数

```
/**
 * 配置中断优先级分组：抢占优先级和子优先级
 * 形参如下：
 * @arg NVIC_PriorityGroup_0: 0bit for 抢占优先级
 *                               4 bits for 子优先级
 * @arg NVIC_PriorityGroup_1: 1 bit for 抢占优先级
 *                               3 bits for 子优先级
 * @arg NVIC_PriorityGroup_2: 2 bit for 抢占优先级
 *                               2 bits for 子优先级
 * @arg NVIC_PriorityGroup_3: 3 bit for 抢占优先级
 *                               1 bits for 子优先级
 * @arg NVIC_PriorityGroup_4: 4 bit for 抢占优先级
 *                               0 bits for 子优先级
 * @ 注意 如果优先级分组为 0，则抢占优先级就不存在，优先级就全部由子优先级控制
 */
void NVIC_PriorityGroupConfig(uint32_t PriorityGroup)
```

(下页继续)

(续上页)

```
{  
    // 检查参数  
    assert_param(IS_NVIC_PRIORITY_GROUP(PriorityGroup));  
    // 设置优先级分组  
    NVIC_SetPriorityGrouping(PriorityGroup);  
}
```

表 16-6 优先级分组真值表

优先级分组	主优先级	子优先级	描述
NVIC_PriorityGroup_0	0	0-15	主-0bit, 子-4bit
NVIC_PriorityGroup_1	0-1	0-7	主-1bit, 子-3bit
NVIC_PriorityGroup_2	0-3	0-3	主-2bit, 子-2bit
NVIC_PriorityGroup_3	0-7	0-1	主-3bit, 子-1bit
NVIC_PriorityGroup_4	0-15	0	主-4bit, 子-0bit

15.4 中断编程

在配置每个中断的时候一般有 3 个编程要点：

- 1、使用 HAL_NVIC_SetPriorityGrouping(uint32_t PriorityGroup) 函数配置中断优先级分组。一般默认是 NVIC_PRIORITYGROUP_4 分组 4。
- 2、使用 HAL_NVIC_SetPriority (IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority) 函数配置具体外设中断通道的抢占优先级和子优先级。
- 3、使用 HAL_NVIC_EnableIRQ 函数使能中断请求。

列表 3: 代码 16-3 IRQn_Type 中断源结构体

```
typedef enum IRQn {  
    //Cortex-M4 处理器异常编号  
    NonMaskableInt_IRQn = -14,
```

(下页继续)

(续上页)

```
MemoryManagement_IRQn    = -12,
BusFault_IRQn             = -11,
UsageFault_IRQn           = -10,
SVCall_IRQn               = -5,
DebugMonitor_IRQn         = -4,
PendSV_IRQn               = -2,
SysTick_IRQn              = -1,
//STM32 外部中断编号
WWDG_IRQn                 = 0,
PVD_IRQn                  = 1,
TAMP_STAMP_IRQn           = 2,

// 限于篇幅，中间部分代码省略，具体的可查看库文件 stm32f4xx.h

CRYP_IRQn                  = 79,
HASH_RNG_IRQn              = 80,
FPU_IRQn                   = 81,
} IRQn_Type;
```

1) **PreemptPriority**: 抢占优先级，具体的值要根据优先级分组来确定，具体参考表 16-6 优先级分组真值表。

2) **SubPriority**: 子优先级，具体的值要根据优先级分组来确定，具体参考表 16-6 优先级分组真值表。

4、编写中断服务函数在启动文件 `startup_stm32f40xx.s` 中我们预先为每个中断都写了一个中断服务函数，只是这些中断函数都是为空，为的只是初始化中断向量表。实际的中断服务函数都需要我们重新编写，中断服务函数我们统一写在 `stm32f4xx_it.c` 这个库文件中。

关于中断服务函数的函数名必须跟启动文件里面预先设置的一样，如果写错，系统就在中断向量表中找不到中断服务函数的入口，直接跳转到启动文件里面预先写好的空函数，并且在里面无限循环，实现不了中断。

第 16 章 EXTI—外部中断/事件控制器

本章参考资料：《STM32F4xx 中文参考手册》系统配置控制器以及中断和事件章节。

上一章我们已经详细介绍了 NVIC，对 STM32F4xx 中断管理系统有个全局的了解，我们这章的内容是 NVIC 的实例应用，也是 STM32F4xx 控制器非常重要的一个资源。学习本章时，配合《STM32F4xx 中文参考手册》系统配置控制器以及中断和事件章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

特别说明，本书内容是以 STM32F40xxx 系列控制器资源讲解。

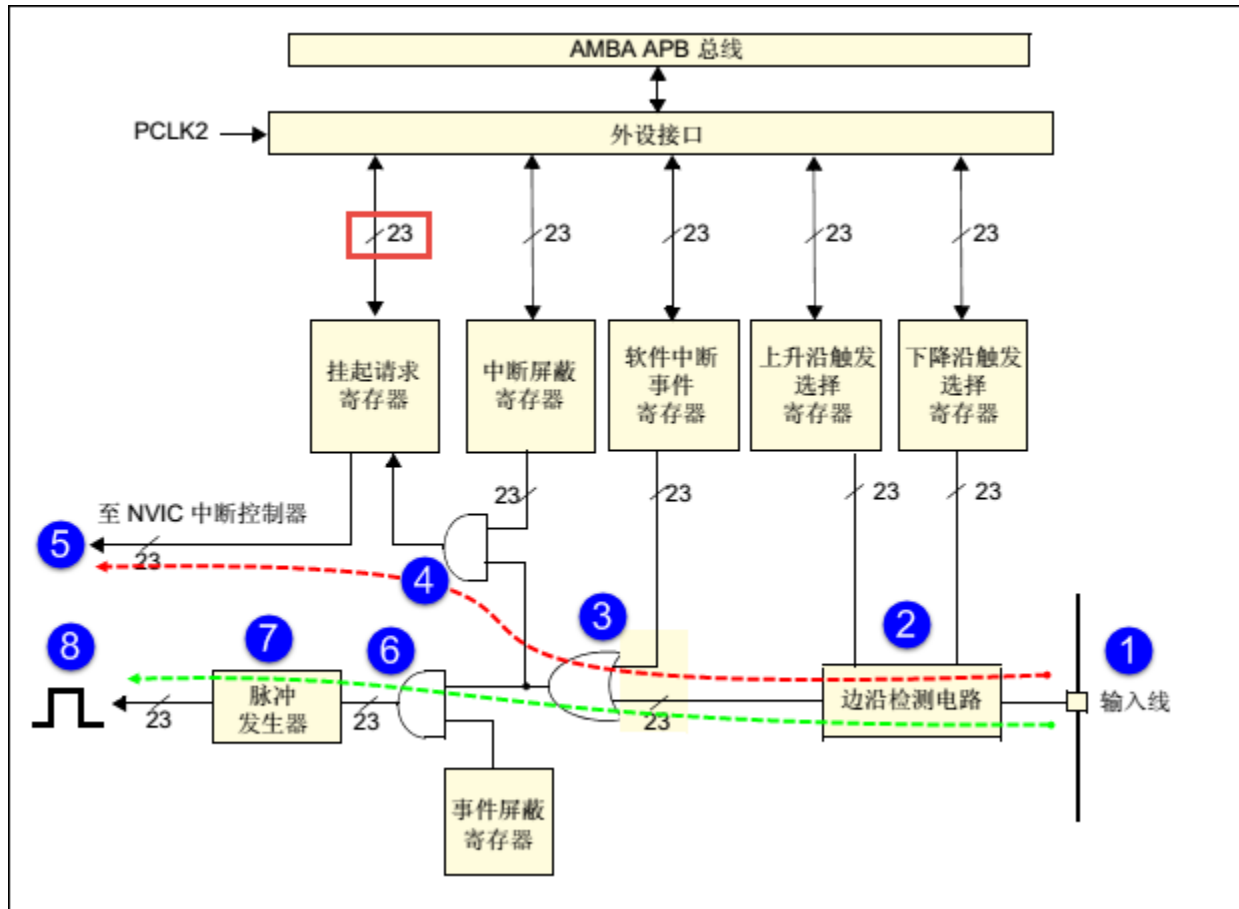
16.1 EXTI 简介

外部中断/事件控制器 (EXTI) 管理了控制器的 23 个中断/事件线。每个中断/事件线都对应有一个边沿检测器，可以实现输入信号的上升沿检测和下降沿的检测。EXTI 可以实现对每个中断/事件线进行单独配置，可以单独配置为中断或者事件，以及触发事件的属性。

16.2 EXTI 功能框图

EXTI 的功能框图包含了 EXTI 最核心内容，掌握了功能框图，对 EXTI 就有一个整体的把握，在编程时思路就非常清晰。EXTI 功能框图见图 17_1。

在图 17_1 可以看到很多在信号线上打一个斜杠并标注“23”字样，这个表示在控制器内部类似的信号线路有 23 个，这与 EXTI 总共有 23 个中断/事件线是吻合的。所以我们只要明白其中一个的原理，那其他 22 个线路原理也就知道了。



EXTI 可分为两大部分功能，一个是产生中断，另一个是产生事件，这两个功能从硬件上就有所不同。

首先我们来看图 17_1 中红色虚线指示的电路流程。它是一个产生中断的线路，最终信号流入到 NVIC 控制器内。

编号 1 是输入线，EXTI 控制器有 23 个中断/事件输入线，这些输入线可以通过寄存器设置为任意一个 GPIO，也可以是一些外设的事件，这部分内容我们将在后面专门讲解。输入线一般是存在电平变化的信号。

编号 2 是一个边沿检测电路，它会根据上升沿触发选择寄存器 (EXTI_RTSR) 和下降沿触发选择寄存器 (EXTI_FTSR) 对应位的设置来控制信号触发。边沿检测电路以输入线作为信号输入端，如果检测到有边沿跳变就输出有效信号 1 给编号 3 电路，否则输出无效信号 0。而 EXTI_RTSR 和 EXTI_FTSR 两个寄存器可以控制器需要检测哪些类型的电平跳变过程，可以是只有上升沿触发、

只有下降沿触发或者上升沿和下降沿都触发。

编号 3 电路实际就是一个或门电路，它一个输入来自编号 2 电路，另外一输入来自软件中断事件寄存器 (EXTI_SWIER)。EXTI_SWIER 允许我们通过程序控制就可以启动中断/事件线，这在某些地方非常有用。我们知道或门的作用就是有” 就为 1，所以这两个输入随便一个有有效信号 1 就可以输出 1 给编号 4 和编号 6 电路。

编号 4 电路是一个与门电路，它一个输入编号 3 电路，另外一个输入来自中断屏蔽寄存器 (EXTI_IMR)。与门电路要求输入都为 1 才输出 1，导致的结果如果 EXTI_IMR 设置为 0 时，那不管编号 3 电路的输出信号是 1 还是 0，最终编号 4 电路输出的信号都为 0；如果 EXTI_IMR 设置为 1 时，最终编号 4 电路输出的信号才由编号 3 电路的输出信号决定，这样我们可以简单的控制 EXTI_IMR 来实现是否产生中断的目的。编号 4 电路输出的信号会被保存到挂起寄存器 (EXTI_PR) 内，如果确定编号 4 电路输出为 1 就会把 EXTI_PR 对应位置 1。

编号 5 是将 EXTI_PR 寄存器内容输出到 NVIC 内，从而实现系统中断事件控制。

接下来我们来看看绿色虚线指示的电路流程。它是一个产生事件的线路，最终输出一个脉冲信号。

产生事件线路是在编号 3 电路之后与中断线路有所不同，之前电路都是共用的。编号 6 电路是一个与门，它一个输入编号 3 电路，另外一个输入来自事件屏蔽寄存器 (EXTI_EMR)。如果 EXTI_EMR 设置为 0 时，那不管编号 3 电路的输出信号是 1 还是 0，最终编号 6 电路输出的信号都为 0；如果 EXTI_EMR 设置为 1 时，最终编号 6 电路输出的信号才由编号 3 电路的输出信号决定，这样我们可以简单的控制 EXTI_EMR 来实现是否产生事件的目的。

编号 7 是一个脉冲发生器电路，当它的输入端，即编号 6 电路的输出端，是一个有效信号 1 时就会产生一个脉冲；如果输入端是无效信号就不会输出脉冲。

编号 8 是一个脉冲信号，就是产生事件的线路最终的产物，这个脉冲信号可以给其他外设电路使用，比如定时器 TIM、模拟数字转换器 ADC 等等，这样的脉冲信号一般用来触发 TIM 或者 ADC 开始转换。

产生中断线路目的是把输入信号输入到 NVIC，进一步会运行中断服务函数，实现功能，这样是软件级的。而产生事件线路目的就是传输一个脉冲信号给其他外设使用，并且是电路级别的信号传输，属于硬件级的。

另外，EXTI 是在 APB2 总线上的，在编程时候需要注意到这点。

16.3 中断/事件线

EXTI 有 23 个中断/事件线，每个 GPIO 都可以被设置为输入线，占用 EXTI0 至 EXTI15，还有另外七根用于特定的外设事件，见表 17_1。

七根特定外设中断/事件线由外设触发，具体用法参考《STM32F4xx 中文参考手册》中对外设的具体说明。

表 17-1 EXTI 中断/事件线

中断/事件线	输入源
EXTI0	PX0(X 可为 A,B,C,D,E,F,G,H,I)
EXTI1	PX1(X 可为 A,B,C,D,E,F,G,H,I)
EXTI2	PX2(X 可为 A,B,C,D,E,F,G,H,I)
EXTI3	PX3(X 可为 A,B,C,D,E,F,G,H,I)
EXTI4	PX4(X 可为 A,B,C,D,E,F,G,H,I)
EXTI5	PX5(X 可为 A,B,C,D,E,F,G,H,I)
EXTI6	PX6(X 可为 A,B,C,D,E,F,G,H,I)
EXTI7	PX7(X 可为 A,B,C,D,E,F,G,H,I)
EXTI8	PX8(X 可为 A,B,C,D,E,F,G,H,I)
EXTI9	PX9(X 可为 A,B,C,D,E,F,G,H,I)
EXTI10	PX10(X 可为 A,B,C,D,E,F,G,H,I)
EXTI11	PX11(X 可为 A,B,C,D,E,F,G,H,I)
EXTI12	PX12(X 可为 A,B,C,D,E,F,G,H,I)
EXTI13	PX13(X 可为 A,B,C,D,E,F,G,H,I)
EXTI14	PX14(X 可为 A,B,C,D,E,F,G,H,I)
EXTI15	PX15(X 可为 A,B,C,D,E,F,G,H,I)
EXTI16	PVD 输出
EXTI17	RTC 闹钟事件
EXTI18	USB 唤醒事件
EXTI19	以太网唤醒事件（只适用互联型）
EXTI20	USB OTG HS(在 FS 中配置) 唤醒事件
EXTI21	RTC 入侵和时间戳事件
EXTI22	RTC 唤醒事件

EXTI0 至 EXTI15 用于 GPIO，通过编程控制可以实现任意一个 GPIO 作为 EXTI 的输入源。由表 17_1 可知，EXTI0 可以通过 AFIO 的外部中断配置寄存器 1(AFIO_EXTICR1) 的 EXTI0[3:0] 位选择配置为 PA0、PB0、PC0、PD0、PE0、PF0、PG0、PH0 或者 PI0，见图 17_2。其他 EXTI 线 (EXTI 中断/事件线) 使用配置都是类似的。

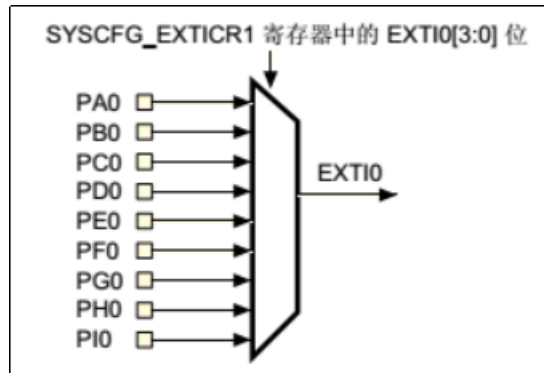


图 17-2 EXTIO 输入源选择

16.4 外部中断控制实验

中断在嵌入式应用中占有非常重要的地位，几乎每个控制器都有中断功能。中断对保证紧急事件得到第一时间处理是非常重要的

我们设计使用外接的按键来作为触发源，使得控制器产生中断，并在中断服务函数中实现控制 RGB 彩灯的任务。

16.4.1 硬件设计

轻触按键在按下时会使得引脚接通，通过电路设计可以使得按下时产生电平变化，见图 17_3。

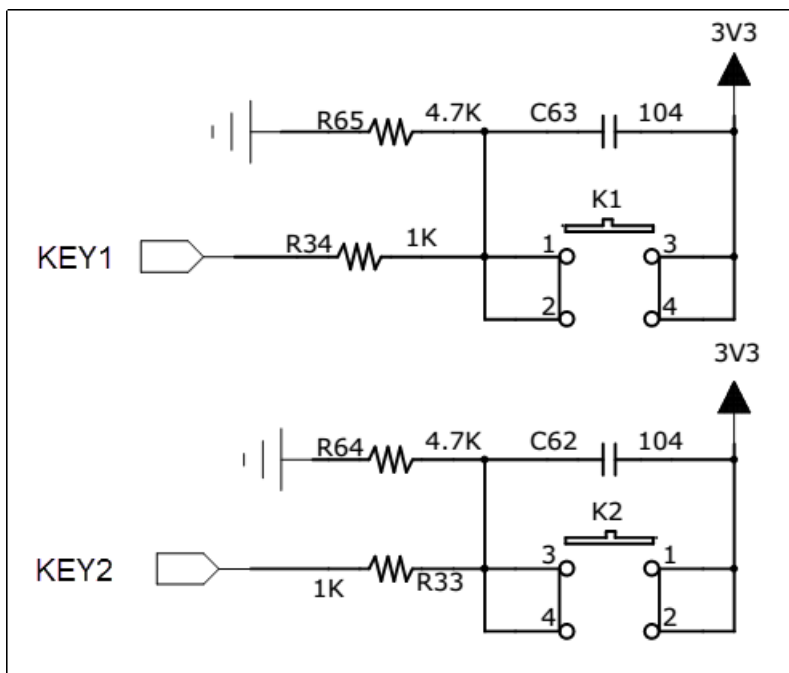


图 17-3 按键电路设计

16.4.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：`bsp_exti.c` 和 `bsp_exti.h` 文件用来存放 EXTI 驱动程序及相关宏定义，中断服务函数放在 `stm32f4xx_it.c` 文件中。

16.4.2.1 编程要点

- 1) 初始化 RGB 彩灯的 GPIO；
- 2) 开启按键 GPIO 时钟；
- 3) 配置 NVIC；
- 4) 配置按键 GPIO 为输入模式；
- 5) 将按键 GPIO 连接到 EXTI 源输入；

6) 配置按键 EXTI 中断/事件线;

7) 编写 EXTI 中断服务函数。

16.4.2.2 软件分析

16.4.2.2.1 按键和 EXTI 宏定义

列表 1: 代码清单 17-1 按键和 EXTI 宏定义

```
//引脚定义
/*****/

#define KEY1_INT_GPIO_PORT          GPIOA
#define KEY1_INT_GPIO_CLK_ENABLE() __GPIOA_CLK_ENABLE();
#define KEY1_INT_GPIO_PIN          GPIO_PIN_0
#define KEY1_INT_EXTI_IRQ           EXTI0_IRQn
#define KEY1_IRQHandler              EXTI0_IRQHandler

#define KEY2_INT_GPIO_PORT          GPIOC
#define KEY2_INT_GPIO_CLK_ENABLE() __GPIOA_CLK_ENABLE();
#define KEY2_INT_GPIO_PIN          GPIO_PIN_13
#define KEY2_INT_EXTI_IRQ           EXTI15_10_IRQn
#define KEY2_IRQHandler              EXTI15_10_IRQHandler

/*****/
```

使用宏定义方法指定与电路设计相关配置，这对于程序移植或升级非常有用的。

16.4.2.2.2 EXTI 中断配置

列表 2: 代码清单 17-2 EXTI 中断配置 (bsp_exit.c 文件)

```
void EXTI_Key_Config(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    /* 开启按键 GPIO 口的时钟 */
    KEY1_INT_GPIO_CLK_ENABLE();
    KEY2_INT_GPIO_CLK_ENABLE();

    /* 选择按键 1 的引脚 */
    GPIO_InitStructure.Pin = KEY1_INT_GPIO_PIN;
    /* 设置引脚为输入模式 */
    GPIO_InitStructure.Mode = GPIO_MODE_IT_RISING;
    /* 设置引脚不上拉也不下拉 */
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    /* 使用上面的结构体初始化按键 */
    HAL_GPIO_Init(KEY1_INT_GPIO_PORT, &GPIO_InitStructure);
    /* 配置 EXTI 中断源 到 key1 引脚、配置中断优先级 */
    HAL_NVIC_SetPriority(KEY1_INT_EXTI_IRQ, 0, 0);
    /* 使能中断 */
    HAL_NVIC_EnableIRQ(KEY1_INT_EXTI_IRQ);

    /* 选择按键 2 的引脚 */
    GPIO_InitStructure.Pin = KEY2_INT_GPIO_PIN;
    /* 配置为下降沿触发 */
    GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
    /* 其他配置与上面相同 */
    HAL_GPIO_Init(KEY2_INT_GPIO_PORT, &GPIO_InitStructure);
    /* 配置 EXTI 中断源 到 key2 引脚、配置中断优先级 */
    HAL_NVIC_SetPriority(KEY2_INT_EXTI_IRQ, 0, 0);
    /* 使能中断 */
}
```

(下页继续)

(续上页)

```
HAL_NVIC_EnableIRQ(KEY2_INT_EXTI_IRQ);  
}
```

首先，使用 `GPIO_InitTypeDef` 结构体定义用于 GPIO 初始化配置的变量，关于这个结构体前面都已经做了详细的讲解。

使用 GPIO 之前必须开启 GPIO 端口的时钟；

调用 `HAL_NVIC_SetPriority` 和 `HAL_NVIC_EnableIRQ` 函数完成对按键 1、按键 2 优先级配置并使能中断通道。

作为中断/时间输入线把 GPIO 配置为中断上升沿触发模式，这里不使用上拉或下拉，有外部电路完全决定引脚的状态。

我们的目的是产生中断，执行中断服务函数，EXTI 选择中断模式，按键 1 使用上升沿触发方式，并使能 EXTI 线。

按键 2 基本上采用与按键 1 相关参数配置，只是改为下降沿触发方式。

16.4.2.2.3 EXTI 中断服务函数

列表 3: 代码清单 17-3 EXTI 中断服务函数

```
void KEY1_IRQHandler(void)  
{  
    //确保是否产生了 EXTI Line 中断  
    if (__HAL_GPIO_EXTI_GET_IT(KEY1_INT_GPIO_PIN) != RESET) {  
        // LED1 取反  
        LED1_TOGGLE;  
        //清除中断标志位  
        __HAL_GPIO_EXTI_CLEAR_IT(KEY1_INT_GPIO_PIN);  
    }  
}
```

(下页继续)

(续上页)

```
void KEY2_IRQHandler(void)
{
    //确保是否产生了 EXTI Line 中断
    if (__HAL_GPIO_EXTI_GET_IT(KEY2_INT_GPIO_PIN) != RESET) {
        // LED2 取反
        LED2_TOGGLE;
        //清除中断标志位
        __HAL_GPIO_EXTI_CLEAR_IT(KEY2_INT_GPIO_PIN);
    }
}
```

当中断发生时，对应的中断服务函数就会被执行，我们可以在中断服务函数实现一些控制。

一般为确保中断确实发生，我们会在中断服务函数调用中断标志位状态读取函数读取外设中断标志位并判断标志位状态。

`__HAL_GPIO_EXTI_GET_IT` 函数用来获取 EXTI 的中断标志位状态，如果 EXTI 线有中断发生函数返回“SET”否则返回“RESET”。实际上，`__HAL_GPIO_EXTI_GET_IT` 函数是通过读取 EXTI_PR 寄存器值来判断 EXTI 线状态的。

按键 1 的中断服务函数我们让 LED1 翻转其状态，按键 2 的中断服务函数我们让 LED2 翻转其状态。执行任务后需要调用 `__HAL_GPIO_EXTI_CLEAR_IT` 函数清除 EXTI 线的中断标志位。

16.4.2.2.4 主函数

列表 4: 代码清单 17-4 主函数

```
int main(void)
{
    /* 系统时钟初始化成 168 MHz */
    SystemClock_Config();
}
```

(下页继续)

(续上页)

```
/* LED 端口初始化 */
LED_GPIO_Config();

/* 初始化 EXTI 中断，按下按键会触发中断，
 * 触发中断会进入 stm32f4xx_it.c 文件中的函数
 *
 * KEY1_IRQHandler 和 KEY2_IRQHandler，处理中断，反转 LED 灯。
 */
EXTI_Key_Config();

/* 等待中断，由于使用中断方式，CPU 不用轮询按键
 */
while (1) {
}
}
```

主函数非常简单，只有三个任务函数。SystemClock_Config 初始化系统时钟，LED_GPIO_Config 函数定义在 bsp_led.c 文件内，完成 RGB 彩灯的 GPIO 初始化配置。EXTI_Key_Config 函数完成两个按键的 GPIO 和 EXTI 配置。

16.4.3 下载验证

保证开发板相关硬件连接正确，把编译好的程序下载到开发板。此时 LED1、LED2 两盏灯（对于 F407-霸天虎、F429-挑战者开发板，则是一个 RGB 彩色灯）是暗的，如果我们按下开发板上的按键 1，LED1 灯变亮，再按下按键 1，LED1 灯又变暗；如果我们按下开发板上的按键 2 并弹开，LED2 变亮，再按下开发板上的 KEY2 并弹开，LED2 灯又变暗。

第 17 章 SysTick—系统定时器

本章参考资料《ARM Cortex™-M4F 技术参考手册》-4.5 章节 SysTick Timer(STK)，和 4.48 章节 SHPRx，其中 STK 这个章节有 SysTick 的简介和寄存器的详细描述。因为 SysTick 是属于 CM4 内核的外设，有关寄存器的定义和部分库函数都在 core_cm4.h 这个头文件中实现。所以学习 SysTick 的时候可以参考这两个资料，一个是文档，一个是源码。

17.1 SysTick 简介

SysTick—系统定时器是属于 CM4 内核中的一个外设，内嵌在 NVIC 中。系统定时器是一个 24bit 的向下递减的计数器，计数器每计数一次的时间为 1/SYSCLK，一般我们设置系统时钟 SYSCLK 等于 168M。当重装载数值寄存器的值递减到 0 的时候，系统定时器就产生一次中断，以此循环往复。

因为 SysTick 是属于 CM4 内核的外设，所以所有基于 CM4 内核的单片机都具有这个系统定时器，使得软件在 CM4 单片机中可以很容易的移植。系统定时器一般用于操作系统，用于产生时基，维持操作系统的心跳。

17.2 SysTick 寄存器介绍

SysTick—系统定时有 4 个寄存器，简要介绍如下。在使用 SysTick 产生定时的时候，只需要配置前三个寄存器，最后一个校准寄存器不需要使用。

表 18-1 SysTick 寄存器汇总

寄存器名称	寄存器描述
CTRL	SysTick 控制及状态寄存器
LOAD	SysTick 重装载数值寄存器
VAL	SysTick 当前数值寄存器
CALIB	SysTick 校准数值寄存器

表 18-2 SysTick 控制及状态寄存器

位段	名称	类型	复位值	描述
16	COUNT-FLAG	R/W	0	如果在上次读取本寄存器后，SysTick 已经计到了 0，则该位为 1。
2	CLK-SOURCE	R/W	0	时钟源选择位，0=AHB/8，1= 处理器时钟 AHB
1	TICK-INT	R/W	0	1=SysTick 倒数计数到 0 时产生 SysTick 异常请求，0= 数到 0 时无动作。也可以通过读取 COUNTFLAG 标志位来确定计数器是否递减到 0
0	EN-ABLE	R/W	0	SysTick 定时器的使能位

表 18-3 SysTick 重装载数值寄存器

位段	名称	类型	复位值	描述
23:0	RELOAD	R/W	0	当倒数计数至零时，将被重装载的值

表 18-4 SysTick 当前数值寄存器

位段	名称	类型	复位值	描述
23:0	CURRENT	R/W	0	读取时返回当前倒计数的值，写它则使之清零，同时还会清除在 SysTick 控制及状态寄存器中的 COUNTFLAG 标志

表 18-5 SysTick 校准数值寄存器

位段	名称	类型	复位值	描述
31	NOREF	R	0	NOREF flag. Reads as zero. Indicates that a separate reference clock is provided. The frequency of this clock is HCLK/8
30	SKEW	R	1	Reads as one. Calibration value for the 1 ms inexact timing is not known because TENMS is not known. This can affect the suitability of SysTick as a software real time clock
23:0	TENMS	R	0	Indicates the calibration value when the SysTick counter runs on HCLK max/8 as external clock. The value is product dependent, please refer to the Product Reference Manual, SysTick Calibration Value section. When HCLK is programmed at the maximum frequency, the SysTick period is 1ms. If calibration information is not known, calculate the calibration value required from the frequency of the processor clock or external clock.

系统定时器的校准数值寄存器在定时实验中不需要用到。有关各个位的描述这里引用手册里面的英文版本，比较晦涩难懂，暂时不知道这个寄存器用来干什么。有研究过的朋友可以交流，起个抛砖引玉的作用。

17.3 SysTick 定时实验

利用 SysTick 产生 1s 的时基，LED 以 1s 的频率闪烁。

17.3.1 硬件设计

SysTick 属于单片机内部的外设，不需要额外的硬件电路，剩下的只需一个 LED 灯即可。

17.3.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_SysTick.c 和 bsp_SysTick.h 文件用来存放 SysTick 驱动程序及相关宏定义，中断服务函数放在 stm32f4xx_it.c 文件中。

17.3.2.1 编程要点

- 1、设置重装载寄存器的值
- 2、清除当前数值寄存器的值
- 3、配置控制与状态寄存器

17.3.2.2 代码分析

SysTick 属于内核的外设，有关的寄存器定义和库函数都在内核相关的库文件 core_cm4.h 中。

17.3.2.2.1 SysTick 配置库函数

列表 1: 代码 18-1 SysTick 配置库函数

```
__STATIC_INLINE uint32_t SysTick_Config(uint32_t ticks)
{
    // 不可能的重装载值, 超出范围
    if ((ticks - 1UL) > SysTick_LOAD_RELOAD_Msk) {
        return (1UL);
    }

    // 设置重装载寄存器
    SysTick->LOAD = (uint32_t)(ticks - 1UL);

    // 设置中断优先级
    NVIC_SetPriority (SysTick_IRQn, (1UL << __NVIC_PRIO_BITS) - 1UL);

    // 设置当前数值寄存器
    SysTick->VAL = 0UL;

    // 设置系统定时器的时钟源为 AHBCLK=168M
    // 使能系统定时器中断
    // 使能定时器
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk |
                    SysTick_CTRL_ENABLE_Msk;

    return (0UL);
}
```

用固件库编程的时候我们只需要调用库函数 SysTick_Config() 即可，形参 ticks 用来设置重装载寄存器的值，最大不能超过重装载寄存器的值 2^{24} ，当重装载寄存器的值递减到 0 的时候产生中断，然后重装载寄存器的值又重新装载往下递减计数，以此循环往复。紧随其后设置好中断优先级，最后配置系统定时器的时钟为 168M，使能定时器和定时器中断，这样系统定时器就配置好了，

一个库函数搞定。

SysTick_Config() 库函数主要配置了 SysTick 中的三个寄存器：LOAD、VAL 和 CTRL，有关具体的部分看代码注释即可。其中还调用了函数库函数 NVIC_SetPriority() 来配置系统定时器的中断优先级，该库函数也在 core_m4.h 中定义，原型如下：

```
__STATIC_INLINE void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)
{
    if ((int32_t) IRQn < 0) {
        SCB->SHP[(((uint32_t) (int32_t) IRQn) & 0xFUL)-4UL] =
            (uint8_t)((priority << (8 - __NVIC_PRIO_BITS)) & (uint32_t) 0xFFUL);
    } else {
        NVIC->IP[(((uint32_t) (int32_t) IRQn)] =
            (uint8_t)((priority << (8 - __NVIC_PRIO_BITS)) & (uint32_t) 0xFFUL);
    }
}
```

因为 SysTick 属于内核外设，跟普通外设的中断优先级有些区别，并没有抢占优先级和子优先级的说法。在 STM32F407 中，内核外设的中断优先级由内核 SCB 这个外设的寄存器：SHPRx (x=1.2.3) 来配置。有关 SHPRx 寄存器的详细描述可参考《Cortex-M4 内核编程手册》4.4.8 章节。下面我们简单介绍下这个寄存器。

SPRH1-SPRH3 是一个 32 位的寄存器，但是只能通过字节访问，每 8 个字段控制着一个内核外设的中断优先级的配置。在 STM32F407 中，只有位 7:4 这高四位有效，低四位没有用到，所以内核外设的中断优先级可编程为：0~15，只有 16 个可编程优先级，数值越小，优先级越高。如果软件优先级配置相同，那就根据他们在中断向量表里面的位置编号来决定优先级大小，编号越小，优先级越高。

表 18-6 系统异常优先级字段

异常	字段	寄存器描述
Memory management fault	PRI_4	SHPR1
Bus fault	PRI_5	
Usage fault	PRI_6	
SVCall	PRI_11	SHPR2
PendSV	PRI_14	SHPR3
SysTick	PRI_15	

如果要修改内核外设的优先级，只需要修改下面三个寄存器对应的某个字段即可。

System handler priority register 1 (SHPR1)

Address offset: 0x18

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								PRI_6[7:4]				PRI_6[3:0]			
								rw	rw	rw	rw	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRI_5[7:4]				PRI_5[3:0]				PRI_4[7:4]				PRI_4[3:0]			
rw	rw	rw	rw	r	r	r	r	rw	rw	rw	rw	r	r	r	r

图 18-1 SHPR1 寄存器

System handler priority register 2 (SHPR2)

Address offset: 0x1C

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRI_11[7:4]				PRI_11[3:0]				Reserved							
rw	rw	rw	rw	r	r	r	r								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

图 18-2 SHPR2 寄存器

System handler priority register 3 (SHPR3)

Address: 0xE000 ED20

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRI_15[7:4]				PRI_15[3:0]				PRI_14[7:4]				PRI_14[3:0]			
rw	rw	rw	rw	r	r	r	r	rw	rw	rw	rw	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

图 18-3 SHPR3 寄存器

在系统定时器中，配置优先级为 $(1UL \ll _NVIC_PRIO_BITS) - 1UL$ ，其中宏 `\_NVIC\_PRIO\_BITS` 为 4，那计算结果就等于 15，可以看出系统定时器此时设置的优先级在内核外设中是最低的，如果要修改优先级则修改这个值即可，范围为：0~15。

```
// 设置系统定时器中断优先级
```

```
NVIC_SetPriority (SysTick_IRQn, (1UL << \_NVIC\_PRIO\_BITS) - 1UL);
```

17.3.2.2.2 SysTick 初始化函数

列表 2: 代码 18-2 SysTick 初始化函数

```
/*
 * @brief 启动系统滴答定时器 SysTick
 *
 * @param 无
 *
 * @retval 无
```

(下页继续)

(续上页)

```
*/  
  
void SysTick_Init(void)  
{  
  
    /* SystemFrequency / 1000 1ms 中断一次  
  
    * SystemFrequency / 100000 10us 中断一次  
  
    * SystemFrequency / 1000000 1us 中断一次  
  
    */  
  
    if (HAL_SYSTICK_Config(SystemCoreClock / 100000)) {  
  
        /* Capture error */  
  
        while (1);  
  
    }  
  
}
```

SysTick 初始化函数由用户编写，里面调用了 SysTick_Config() 这个固件库函数，通过设置该固件库函数的形参，就决定了系统定时器经过多少时间就产生一次中断。

17.3.2.2.3 SysTick 中断时间的计算

SysTick 定时器的计数器是向下递减计数的，计数一次的时间 $T_{DEC}=1/CLK_{AHB}$ ，当重装载寄存器中的值 $VALUE_{LOAD}$ 减到 0 的时候，产生中断，可知中断一次的时间 $T_{INT}=VALUE_{LOAD} * T_{DEC}=VALUE_{LOAD}/CLK_{AHB}$ ，以 F407 霸天虎为例，其中 $CLK_{AHB}=168MHz$ 。如果设置 $VALUE_{LOAD}$ 为 168，那中断一次的时间 $T_{INT}=168/168M=1\mu s$ 。不过 $1\mu s$ 的中断没啥意义，整个程序的重心都花在进出中断上了，根本没有时间处理其他的任务。

```
SysTick_Config(SystemCoreClock / 1000000)
```

`SysTick_Config()` 的形参我们配置为 $SystemCoreClock / 1000000=168M/1000000=1680$ ，从刚刚分析我们知道这个形参的值最终是写到重装载寄存器 `LOAD` 中的，从而可知我们现在把 SysTick 定时器中断一次的时间 $T_{INT}=1680/168M=10\mu s$ 。

17.3.2.2.4 SysTick 定时时间的计算

当设置好中断时间 T_{INT} 后，我们可以设置一个变量 `t`，用来记录进入中断的次数，那么变量 `t` 乘以中断的时间 T_{INT} 就可以计算出需要定时的时间。

17.3.2.2.5 SysTick 定时函数

现在我们定义一个微秒级别的延时函数，形参为 `nTime`，当用这个形参乘以中断时间 T_{INT} 就得出我们需要的延时时间，其中 T_{INT} 我们已经设置好为 $10\mu s$ 。关于这个函数的具体调用看注释即可。

```
/*  
  
* @brief us 延时程序,10us 为一个单位  
  
* @param  
  
* @arg nTime: Delay_us( 1 ) 则实现的延时为 1 * 10us = 10us
```

(下页继续)

(续上页)

```
* @retval 无  
*/  
  
void Delay_us(__IO u32 nTime)  
{  
  
TimingDelay = nTime;  
  
while (TimingDelay != 0);  
  
}
```

函数 Delay_us() 中我们等待 TimingDelay 为 0，当 TimingDelay 为 0 的时候表示延时时间到。变量 TimingDelay 在中断函数中递减，即 SysTick 每进一次中断即 10us 的时间 TimingDelay 递减一次。

17.3.2.2.6 SysTick 中断服务函数

```
void SysTick_Handler(void)  
{  
  
TimingDelay_Decrement();  
  
}
```

中断复位函数调用了另外一个函数 TimingDelay_Decrement()，原型如下：

```
/*  
  
* @brief 获取节拍程序  
  
* @param 无  
  
* @retval 无  
  
* @attention 在 SysTick 中断函数 SysTick_Handler() 调用  
  
*/  
  
void TimingDelay_Decrement(void)  
{  
  
if (TimingDelay != 0x00) {  
  
    TimingDelay--;  
  
}  
  
}
```

TimingDelay 的值等于延时函数中传进去的 nTime 的值，比如 nTime=100000，则延时的时间等于 100000*10us=1s。

17.3.2.2.7 主函数

```
int main(void)

{

    /* 系统时钟初始化成 168MHz */

    SystemClock_Config();

    /* LED 端口初始化 */

    LED_GPIO_Config();

    /* 配置 SysTick 为 10us 中断一次，

    时间到后触发定时中断，

    * 进入 stm32f4xx_it.

    c 文件的 SysTick_Handler 处理，通过数中断次数计时

    */

    SysTick_Init();

    while (1) {

        LED_RED;

        Delay_us(100000); // 10000 * 10us = 1000ms
```

(下页继续)

(续上页)

```
    LED_GREEN;

    Delay_us(100000); // 10000 * 10us = 1000ms

    LED_BLUE;

    Delay_us(100000); // 10000 * 10us = 1000ms

}

}
```

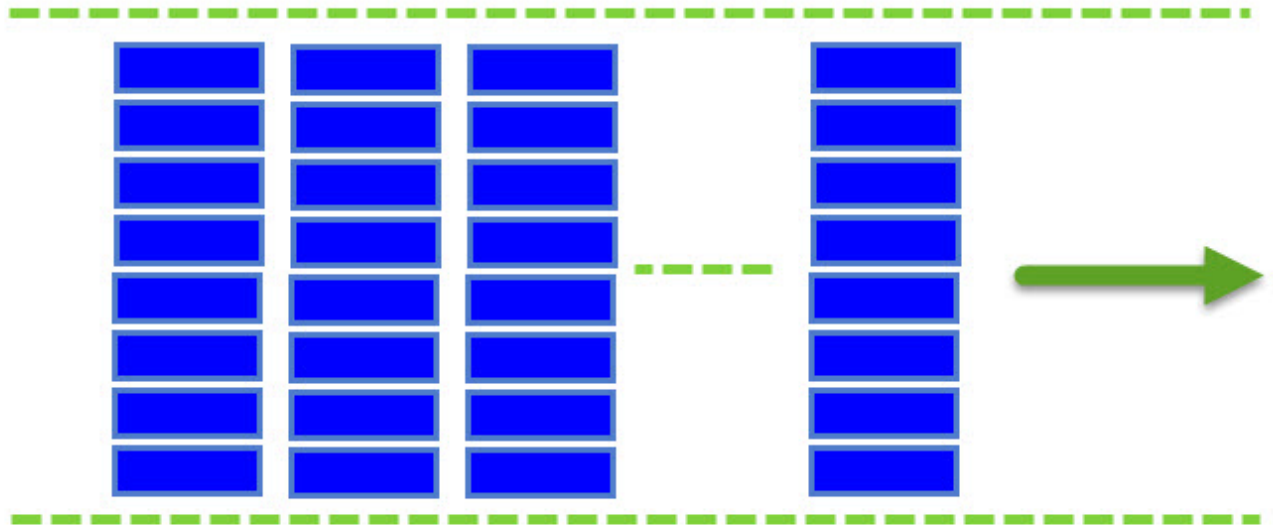
主函数中初始化了 LED 和 SysTick，然后在一个 while 循环中以 1s 的频率让 LED 闪烁。

第 18 章 通讯的基本概念

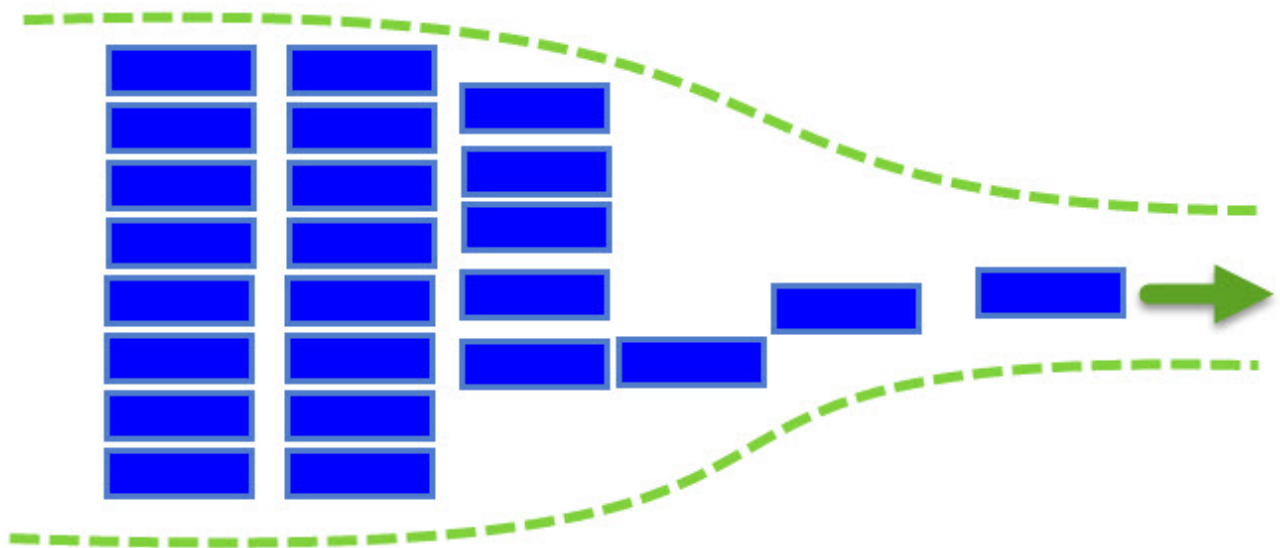
在计算机设备与设备之间或集成电路之间常常需要进行数据传输，在本书后面的章节中我们会学习到各种各样的通讯方式，所以在本章中我们先统一介绍这些通讯的基本概念。

18.1 串行通讯与并行通讯

按数据传送的方式，通讯可分为串行通讯与并行通讯，串行通讯是指设备之间通过少量数据信号线（一般是 8 根以下），地线以及控制信号线，按数据位形式一位一位地传输数据的通讯方式。而并行通讯一般是指使用 8、16、32 及 64 根或更多的数据线进行传输的通讯方式，它们的通讯传输对比说明见图 19_1，并行通讯就像多个车道的公路，可以同时传输多个数据位的数据，而串行通讯，而串行通讯就像单个车道的公路，同一时刻只能传输一个数据位的数据。



并行通讯



串行通讯

图 19-1 并行通讯与串行通讯的对比图

很明显，因为一次可传输多个数据位的数据，在数据传输速率相同的情况下，并行通讯传输的数据量要大得多，而串行通讯则可以节省数据线的硬件成本（特别是远距离时）以及 PCB 的布线面积，串行通讯与并行通讯的特性对比见表 19-1。

表 19-1 串行通讯与并行通讯的特性对比

特性	串行通讯	并行通讯
通讯距离	较远	较近
抗干扰能力	较强	较弱
传输速率	较慢	较高
成本	较低	较高

不过由于并行传输对同步要求较高，且随着通讯速率的提高，信号干扰的问题会显著影响通讯性能，现在随着技术的发展，越来越多的应用场合采用高速率的串行差分传输。

18.2 全双工、半双工及单工通讯

根据数据通讯的方向，通讯又分为全双工、半双工及单工通讯，它们主要以信道的方向来区分，见图 19-2 及表 19-2。

表 19-2 通讯方式说明

通讯方式	说明
全双工	在同一时刻，两个设备之间可以同时收发数据
半双工	两个设备之间可以收发数据，但不能在同一时刻进行
单工	在任何时刻都只能进行一个方向的通讯，即一个固定为发送设备，另一个固定为接收设备

仍以公路来类比，全双工的通讯就是一个双向车道，两个方向上的车流互不相干；半双工则像乡间小道那样，同一时刻只能让一辆小车通过，另一方向的来车只能等待道路空出来时才能经过；而单工则像单行道，另一方向的车辆完全禁止通行。

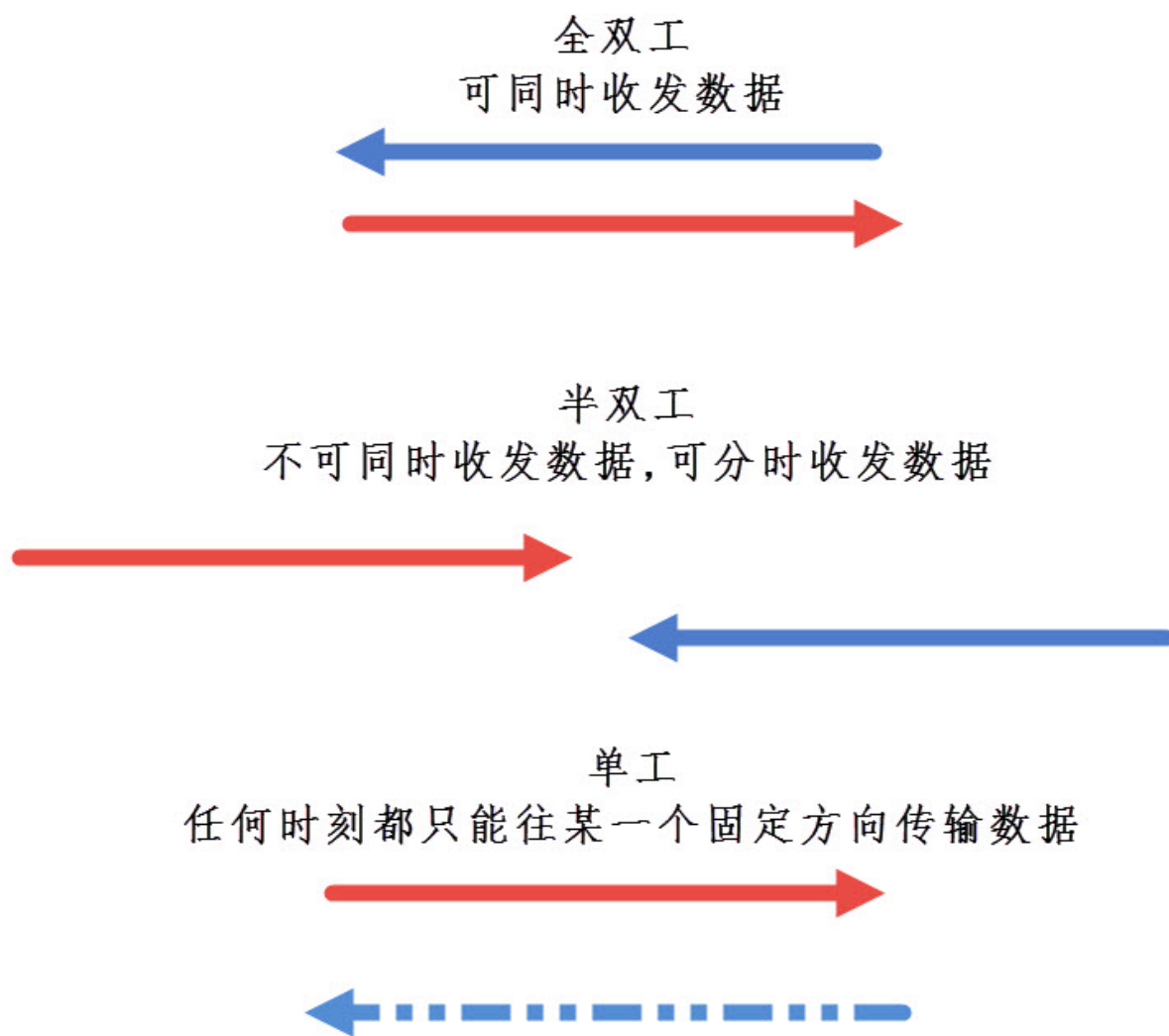


图 19-2 全双工、半双工及单工通讯

18.3 同步通讯与异步通讯

根据通讯的数据同步方式，又分为同步和异步两种，可以根据通讯过程中是否有使用到时钟信号进行简单的区分。

在同步通讯中，收发设备双方会使用一根信号线表示时钟信号，在时钟信号的驱动下双方进行协调，同步数据，见图 19_3。通讯中通常双方会统一规定在时钟信号的上升沿或下降沿对数据线进行采样。

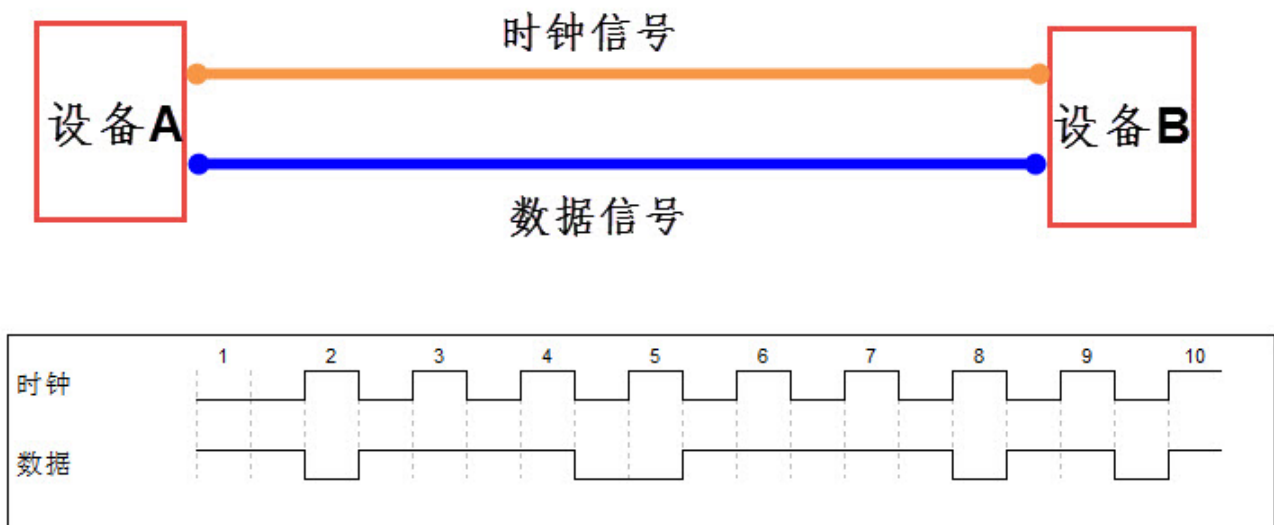


图 19-3 同步通讯

在异步通讯中不使用时钟信号进行数据同步，它们直接在数据信号中穿插一些同步用的信号位，或者把主体数据进行打包，以数据帧的格式传输数据，见图 19_4，某些通讯中还需要双方约定数据的传输速率，以便更好地同步。



图 19-4 某种异步通讯

在同步通讯中，数据信号所传输的内容绝大部分就是有效数据，而异步通讯中会包含有帧的各种标识符，所以同步通讯的效率更高，但是同步通讯双方的时钟允许误差较小，而异步通讯双方的时钟允许误差较大。

18.4 通讯速率

衡量通讯性能的一个非常重要的参数就是通讯速率，通常以比特率 (Bitrate) 来表示，即每秒钟传输的二进制位数，单位为比特每秒 (bit/s)。容易与比特率混淆的概念是“波特率” (Baudrate)，它表示每秒钟传输了多少个码元。而码元是通讯信号调制的概念，通讯中常用时间间隔相同的符号来表示一个二进制数字，这样的信号称为码元。如常见的通讯传输中，用 0V 表示数字 0，5V 表示数字 1，那么一个码元可以表示两种状态 0 和 1，所以一个码元等于一个二进制比特位，此时波特率的大小与比特率一致；如果在通讯传输中，有 0V、2V、4V 以及 6V 分别表示二进制数 00、01、10、11，那么每个码元可以表示四种状态，即两个二进制比特位，所以码元数是二进制比特位数的一半，这个时候的波特率为比特率的一半。因为很多常见的通讯中一个码元都是表示两种

状态，人们常常直接以波特率来表示比特率，虽然严格来说没什么错误，但希望您能了解它们的区别。

第 19 章 USART—串口通讯

本章参考资料：《STM32F4xx 中文参考手册》USART 章节。

学习本章时，配合《STM32F4xx 中文参考手册》USART 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

特别说明，本书内容是以 STM32F4xxx 系列控制器资源讲解。

19.1 串口通讯协议简介

串口通讯 (Serial Communication) 是一种设备间非常常用的串行通讯方式，因为它简单便捷，因此大部分电子设备都支持该通讯方式，电子工程师在调试设备时也经常使用该通讯方式输出调试信息。

在计算机科学里，大部分复杂的问题都可以通过分层来简化。如芯片被分为内核层和片上外设；STM32HAL 库则是在寄存器与用户代码之间的软件层。对于通讯协议，我们也以分层的方式来理解，最基本的是把它分为物理层和协议层。物理层规定通讯系统中具有机械、电子功能部分的特性，确保原始数据在物理媒体的传输。协议层主要规定通讯逻辑，统一收发双方的数据打包、解包标准。简单来说物理层规定我们用嘴巴还是用肢体来交流，协议层则规定我们用中文还是英文来交流。

下面我们分别对串口通讯协议的物理层及协议层进行讲解。

19.1.1 物理层

串口通讯的物理层有很多标准及变种，我们主要讲解 RS-232 标准，RS-232 标准主要规定了信号的用途、通讯接口以及信号的电平标准。

使用 RS-232 标准的串口设备间常见的通讯结构见图 20_1。

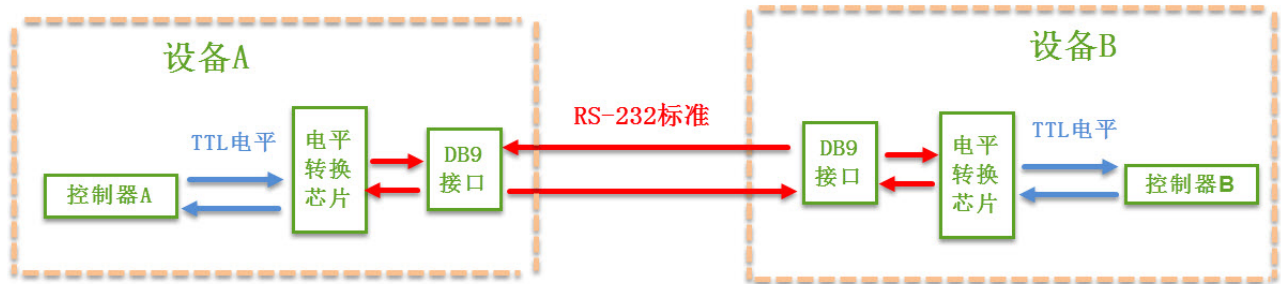


图 20-1 串口通讯结构图

在上面的通讯方式中，两个通讯设备的“DB9 接口”之间通过串口信号线建立起连接，串口信号线中使用“RS-232 标准”传输数据信号。由于 RS-232 电平标准的信号不能被控制器直接识别，所以这些信号会经过一个“电平转换芯片”转换成控制器能识别的“TTL 标准”的电平信号，才能实现通讯。

19.1.1.1 电平标准

根据通讯使用的电平标准不同，串口通讯可分为 TTL 标准及 RS-232 标准，见表 20-1。

表 20-1 TTL 电平标准与 RS232 电平标准

通讯标准	电平标准 (发送端)
5V TTL	逻辑 1: 2.4V-5V
	逻辑 0: 0~0.5V
RS-232	逻辑 1: -15V~-3V
	逻辑 0: +3V~+15V

我们知道常见的电子电路中常使用 TTL 的电平标准，理想状态下，使用 5V 表示二进制逻辑 1，使用 0V 表示逻辑 0；而为了增加串口通讯的远距离传输及抗干扰能力，它使用 -15V 表示逻辑 1，+15V 表示逻辑 0。使用 RS232 与 TTL 电平校准表示同一个信号时的对比见图 20_2。

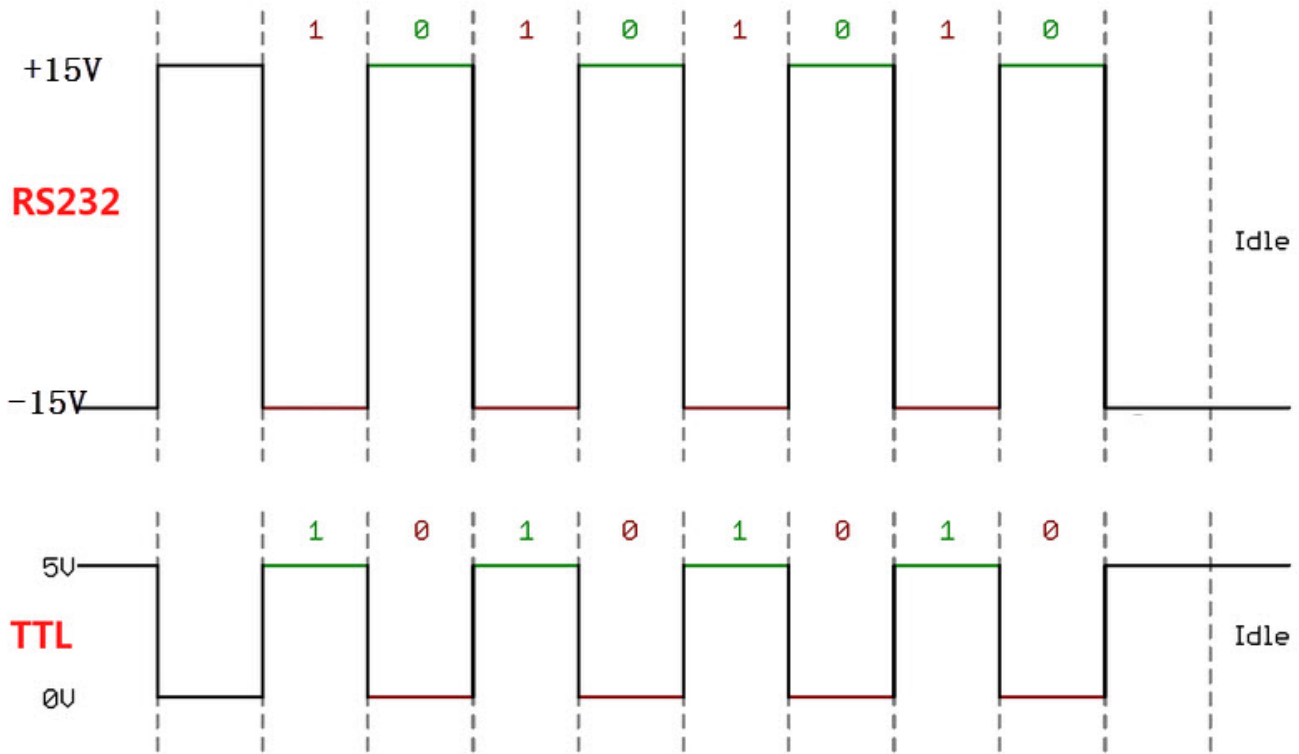


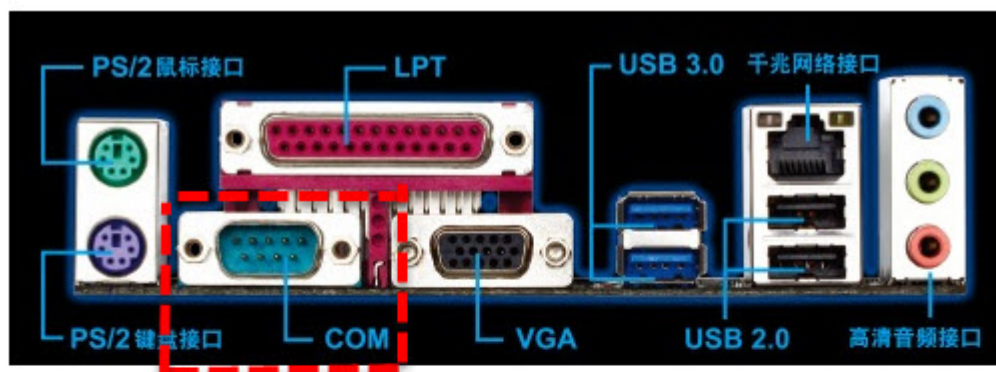
图 20-2 RS-232 与 TTL 电平标准下表示同一个信号

因为控制器一般使用 TTL 电平标准，所以常常会使用 MA3232 芯片对 TTL 及 RS-232 电平的信号进行互相转换。

19.1.1.2 RS-232 信号线

在最初的应用中，RS-232 串口标准常用于计算机、路由与调制解调器 (MODEM，俗称“猫”) 之间的通讯，在这种通讯系统中，设备被分为数据终端设备 DTE(计算机、路由) 和数据通讯设备 DCE(调制解调器)。我们以这种通讯模型讲解它们的信号线连接方式及各个信号线的作用。

在旧式的台式计算机中一般会有 RS-232 标准的 COM 口 (也称 DB9 接口)，见图 20_3。



COM口即DB9接口



串口线

图 20-3 电脑主板上的 COM 口及串口线

其中接线口以针式引出信号线的称为公头，以孔式引出信号线的称为母头。在计算机中一般引出公头接口，而在调制解调器设备中引出的一般为母头，使用上图中的串口线即可把它与计算机连接起来。通讯时，串口线中传输的信号就是使用前面讲解的 RS-232 标准调制的。

在这种应用场合下，DB9 接口中的公头及母头的各个引脚的标准信号线接法见图 20_4 及表 20_2。

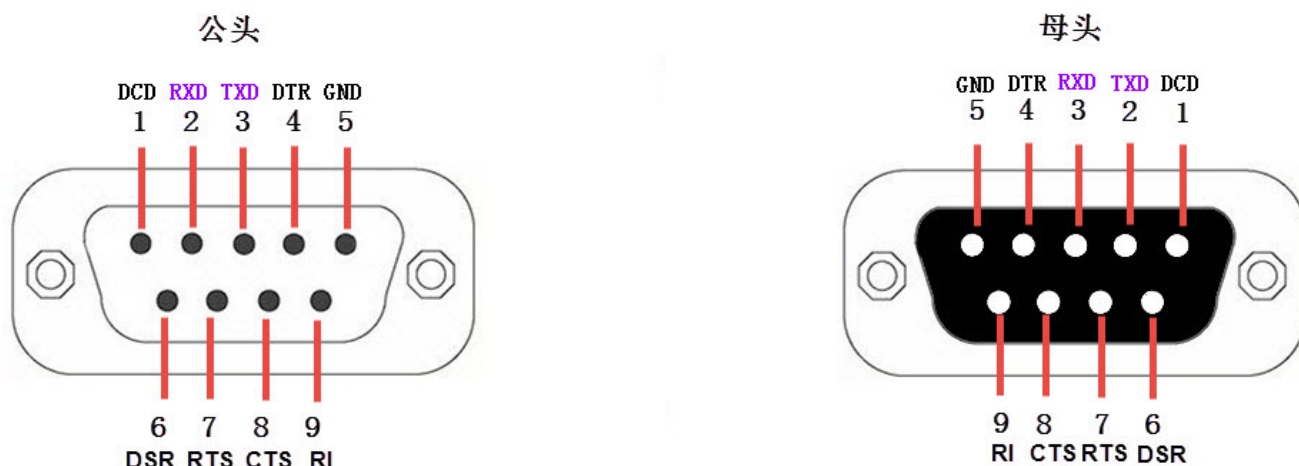


图 20-4 DB9 标准的公头及母头接法

表 20-2 DB9 信号线说明 (公头，为方便理解，可把 DTE 理解为计算机，DCE 理解为调制调解器)

序号	名称	符号	数据方向	说明
1	载波检测	DCD	DTEDCE	Data Carrier Detect, 数据载波检测, 用于 DTE 告知对方, 本机是否收到对方的载波信号
2	接收数据	RXD	DTEDCE	Receive Data, 数据接收信号, 即输入。
3	发送数据	TXD	DTEDCE	Transmit Data, 数据发送信号, 即输出。两个设备之间的 TXD 与 RXD 应交叉相连
4	数据终端 (DTE) 就绪	DTR	DTEDCE	Data Terminal Ready, 数据终端就绪, 用于 DTE 向对方告知本机是否已准备好
5	信号地	GND	.	地线, 两个通讯设备之间的地电位可能不一样, 这会影响收发双方的电平信号, 所以两个串口设备之间必须要使用地线连接, 即共地。
6	数据设备 (DCE) 就绪	DSR	DTEDCE	Data Set Ready, 数据发送就绪, 用于 DCE 告知对方本机是否处于待命状态
论坛: https://www.firebbs.cn/		283	天猫: https://yehuosm.tmall.com	
7	请求发送	RTS	DTEDCE	Request To Send, 请求发送。DTE

上表中的是计算机端的 DB9 公头标准接法，由于两个通讯设备之间的收发信号 (RXD 与 TXD) 应交叉相连，所以调制解调器端的 DB9 母头的收发信号接法一般与公头的相反，两个设备之间连接时，只要使用“直通型”的串口线连接起来即可，见图 20_5。

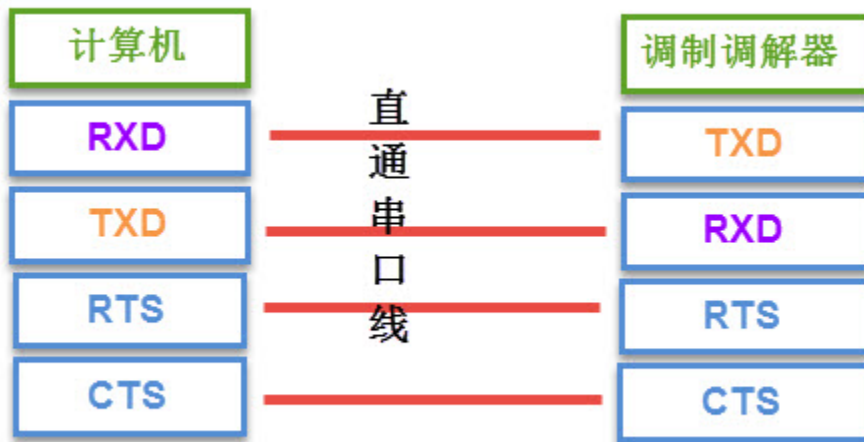


图 20-5 计算机与调制解调器的信号线连接

串口线中的 RTS、CTS、DSR、DTR 及 DCD 信号，使用逻辑 1 表示信号有效，逻辑 0 表示信号无效。例如，当计算机端控制 DTR 信号线表示为逻辑 1 时，它是为了告知远端的调制解调器，本机已准备好接收数据，0 则表示还没准备就绪。

在目前的其它工业控制使用的串口通讯中，一般只使用 RXD、TXD 以及 GND 三条信号线，直接传输数据信号，而 RTS、CTS、DSR、DTR 及 DCD 信号都被裁剪掉了。

19.1.2 协议层

串口通讯的数据包由发送设备通过自身的 TXD 接口传输到接收设备的 RXD 接口。在串口通讯的协议层中，规定了数据包的内容，它由起始位、主体数据、校验位以及停止位组成，通讯双方的数据包格式要约定一致才能正常收发数据，其组成见图 20_6。

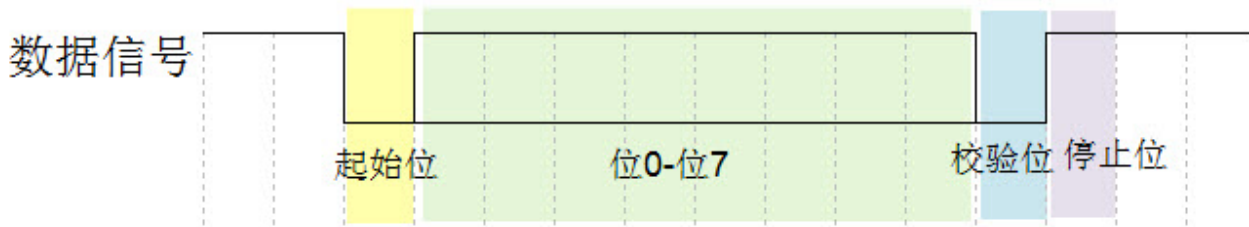


图 20-6 串口数据包的基本组成

19.1.2.1 波特率

本章中主要讲解的是串口异步通讯，异步通讯中由于没有时钟信号（如前面讲解的 DB9 接口中是没有时钟信号的），所以两个通讯设备之间需要约定好波特率，即每个码元的长度，以便对信号进行解码，图 20_6 中用虚线分开的每一格就是代表一个码元。常见的波特率为 4800、9600、115200 等。

19.1.2.2 通讯的起始和停止信号

串口通讯的一个数据包从起始信号开始，直到停止信号结束。数据包的起始信号由一个逻辑 0 的数据位表示，而数据包的停止信号可由 0.5、1、1.5 或 2 个逻辑 1 的数据位表示，只要双方约定一致即可。

19.1.2.3 有效数据

在数据包的起始位之后紧接着的就是要传输的主体数据内容，也称为有效数据，有效数据的长度常被约定为 5、6、7 或 8 位长。

19.1.2.4 数据校验

在有效数据之后，有一个可选的数据校验位。由于数据通信相对更容易受到外部干扰导致传输数据出现偏差，可以在传输过程加上校验位来解决这个问题。校验方法有奇校验 (odd)、偶校验 (even)、0 校验 (space)、1 校验 (mark) 以及无校验 (noparity)，它们介绍如下：

□ 奇校验要求有效数据和校验位中“1”的个数为奇数，比如一个 8 位长的有效数据为：01101001，此时总共有 4 个“1”，为达到奇校验效果，校验位为“1”，最后传输的数据将是 8 位的有效数据加上 1 位的校验位总共 9 位。

□ 偶校验与奇校验要求刚好相反，要求帧数据和校验位中“1”的个数为偶数，比如数据帧：11001010，此时数据帧“1”的个数为 4 个，所以偶校验位为“0”。

□ 0 校验是不管有效数据中的内容是什么，校验位总为“0”，1 校验是校验位总为“1”。

□ 在无校验的情况下，数据包中不包含校验位。

19.2 STM32 的 USART 简介

STM32 芯片具有多个 USART 外设用于串口通讯，它是 Universal Synchronous Asynchronous Receiver and Transmitter 的缩写，即通用同步异步收发器可以灵活地与外部设备进行全双工数据交换。有别于 USART，它还有具有 UART 外设 (Universal Asynchronous Receiver and Transmitter)，它是在 USART 基础上裁剪掉了同步通信功能，只有异步通信。简单区分同步和异步就是看通信时需不需要对外提供时钟输出，我们平时用的串口通信基本都是 UART。

USART 满足外部设备对工业标准 NRZ 异步串行数据格式的要求，并且使用了小数波特率发生器，可以提供多种波特率，使得它的应用更加广泛。USART 支持同步单向通信和半双工单线通信；还支持局域互连网络 LIN、智能卡 (SmartCard) 协议与 IrDA(红外线数据协会) SIR ENDEC 规范。

USART 支持使用 DMA，可实现高速数据通信，有关 DMA 具体应用将在 DMA 章节作具体讲解。

USART 在 STM32 应用最多莫过于“打印”程序信息，一般在硬件设计时都会预留一个 USART 通信接口连接电脑，用于在调试程序是可以把一些调试信息“打印”在电脑端的串口调试助手工具上，从而了解程序运行是否正确、指出运行出错位置等等。

STM32 的 USART 输出的是 TTL 电平信号，若需要 RS-232 标准的信号可使用 MAX3232 芯片进行转换。

19.3 USART 功能框图

STM32 的 USART 的功能框图包含了 USART 最核心内容，掌握了功能框图，对 USART 就有一个整体的把握，在编程时就思路就非常清晰。USART 功能框图见图 20_7。

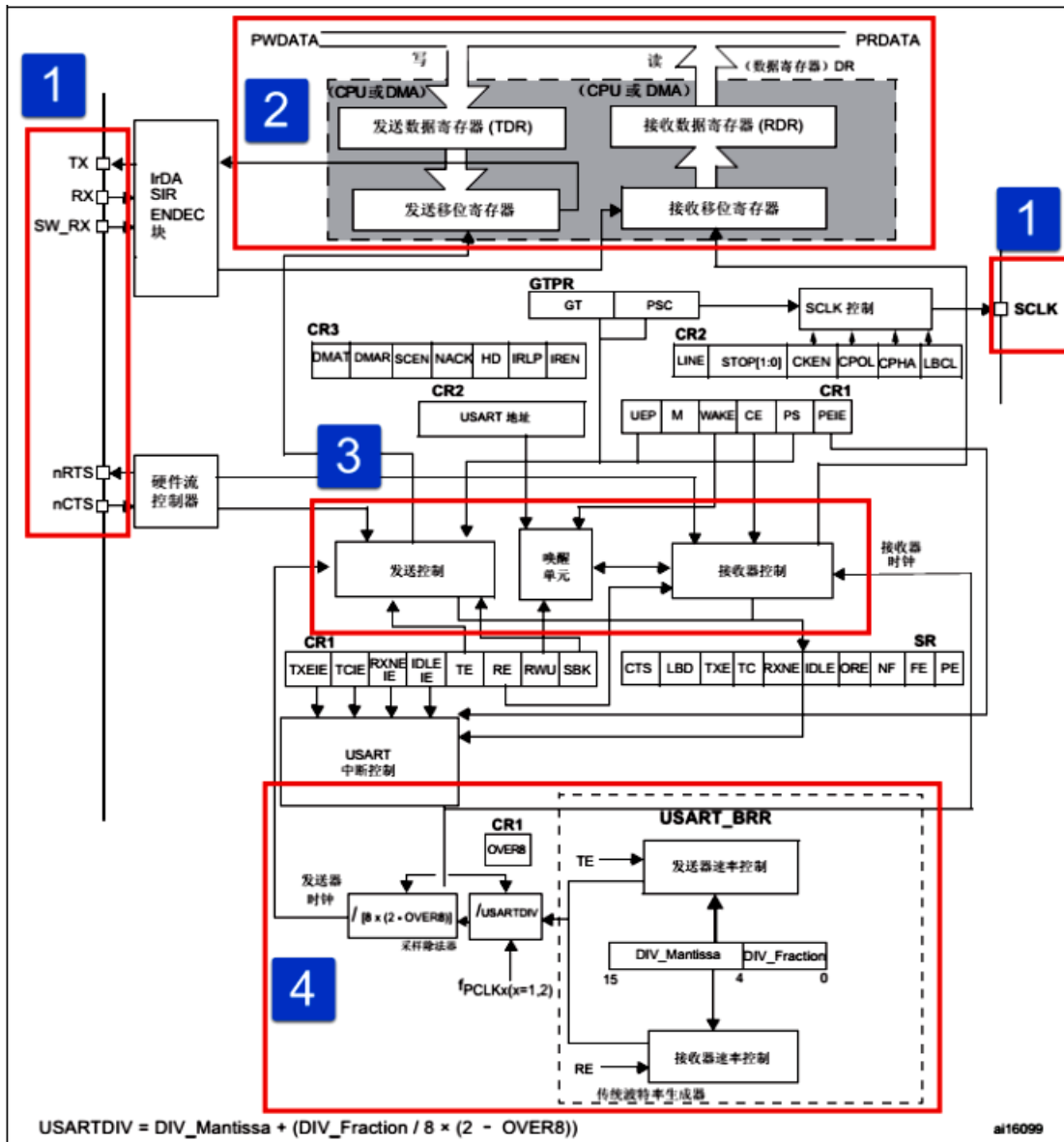


图 20-7 USART 功能框图

19.3.1 □ 功能引脚

TX：发送数据输出引脚。

RX：接收数据输入引脚。

SW_RX：数据接收引脚，只用于单线和智能卡模式，属于内部引脚，没有具体外部引脚。

nRTS：请求以发送 (Request To Send)，n 表示低电平有效。如果使能 RTS 流控制，当 USART 接收器准备好接收新数据时就会将 nRTS 变成低电平；当接收寄存器已满时，nRTS 将被设置为高电平。该引脚只适用于硬件流控制。

nCTS：清除以发送 (Clear To Send)，n 表示低电平有效。如果使能 CTS 流控制，发送器在发送下一帧数据之前会检测 nCTS 引脚，如果为低电平，表示可以发送数据，如果为高电平则在发送完当前数据帧之后停止发送。该引脚只适用于硬件流控制。

SCLK：发送器时钟输出引脚。这个引脚仅适用于同步模式。

USART 引脚在 STM32F407ZGT6 芯片具体发布见[表 20_3](#)。

表 20-3 STM32F407ZGT6 芯片的 USART 引脚

	APB2(最高 84MHz)		APB1(最高 42MHz)			
	USART1	USART6	USART2	USART3	UART4	UART5
TX	PA9/PB6	PC6/PG14	PA2/PD5	PB10/PD8 /PC10	PA0/PC10	PC12
RX	PA10/PB7	PC7/PG9	PA3/PD6	PB11/PD9 /PC11	PA1/PC11	PD2
SCLK	PA8	PG7/PC8	PA4/PD7	PB12/PD10 /PC12	-	-
nCTS	PA11	PG13/PG15	PA0/PD3	PB13/PD11	-	-
nRTS	PA12	PG8/PG12	PA1/PD4	PB14/PD12	-	-

STM32F407ZGT6 有四个 USART 和两个 UART，其中 USART1 和 USART6 的时钟来源于 APB2 总线时钟，其最大频率为 84MHz，其他四个的时钟来源于 APB1 总线时钟，其最大频率为 42MHz。

UART 只是异步传输功能，所以没有 SCLK、nCTS 和 nRTS 功能引脚。观察[表 20_3](#)可发现很多 USART 的功能引脚有多个引脚可选，这非常方便硬件设计，只要在程序编程时软件绑定引脚即可。

19.3.2 □ 数据寄存器

USART 数据寄存器 (USART_DR) 只有低 9 位有效，并且第 9 位数据是否有效要取决于 USART 控制寄存器 1(USART_CR1) 的 M 位设置，当 M 位为 0 时表示 8 位数据字长，当 M 位为 1 表示 9 位数据字长，我们一般使用 8 位数据字长。

USART_DR 包含了已发送的数据或者接收到的数据。USART_DR 实际是包含了两个寄存器，一个专门用于发送的可写 TDR，一个专门用于接收的可读 RDR。当进行发送操作时，往 USART_DR 写入数据会自动存储在 TDR 内；当进行读取操作时，向 USART_DR 读取数据会自动提取 RDR 数据。

TDR 和 RDR 都是介于系统总线和移位寄存器之间。串行通信是一个位一个位传输的，发送时把 TDR 内容转移到发送移位寄存器，然后把移位寄存器数据每一位发送出去，接收时把接收到的每一位顺序保存在接收移位寄存器内然后才转移到 RDR。

USART 支持 DMA 传输，可以实现高速数据传输，具体 DMA 使用将在 DMA 章节讲解。

19.3.3 □ 控制器

USART 有专门控制发送的发送器、控制接收的接收器，还有唤醒单元、中断控制等等。使用 USART 之前需要向 USART_CR1 寄存器的 UE 位置 1 使能 USART，UE 位用来开启供给给串口的时钟。发送或者接收数据字长可选 8 位或 9 位，由 USART_CR1 的 M 位控制。

发送器

当 USART_CR1 寄存器的发送使能位 TE 置 1 时，启动数据发送，发送移位寄存器的数据会在 TX 引脚输出，低位在前，高位在后。如果是同步模式 SCLK 也输出时钟信号。

一个字符帧发送需要三个部分：起始位 + 数据帧 + 停止位。起始位是一个位周期的低电平，位周期就是每一位占用的时间；数据帧就是我们要发送的 8 位或 9 位数据，数据是从最低位开始传输的；停止位是一定时间周期的高电平。

停止位时间长短是可以通过 USART 控制寄存器 2(USART_CR2) 的 STOP[1:0] 位控制，可选 0.5 个、1 个、1.5 个和 2 个停止位。默认使用 1 个停止位。2 个停止位适用于正常 USART 模式、单线模式和调制解调器模式。0.5 个和 1.5 个停止位用于智能卡模式。

当选择 8 位字长，使用 1 个停止位时，具体发送字符时序图见图 20_8。

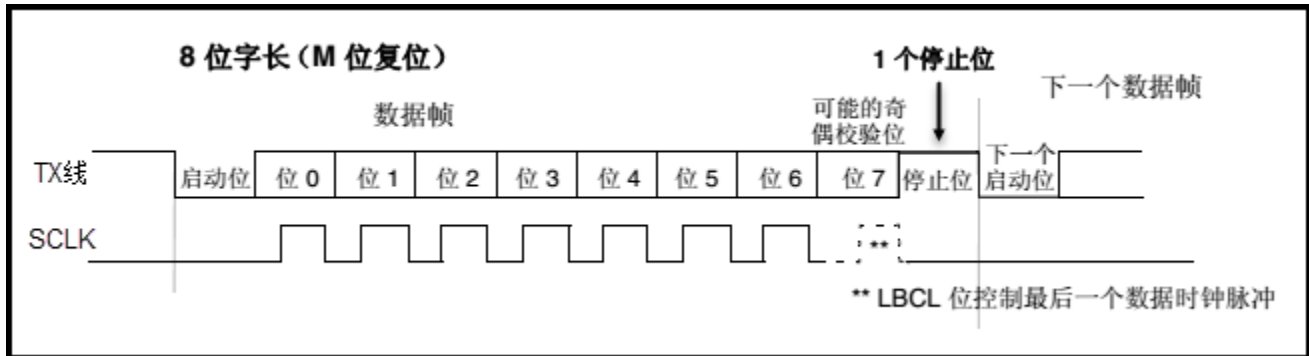


图 20-8 字符发送时序图

当发送使能位 TE 置 1 之后，发送器开始会先发送一个空闲帧 (一个数据帧长度的高电平)，接下来就可以往 USART_DR 寄存器写入要发送的数据。在写入最后一个数据后，需要等待 USART 状态寄存器 (USART_SR) 的 TC 位为 1，表示数据传输完成，如果 USART_CR1 寄存器的 TCIE 位置 1，将产生中断。

在发送数据时，编程的时候有几个比较重要的标志位我们来总结下。

名称	描述
TE	发送使能
TXE	发送寄存器为空，发送单个字节的时候使用
TC	发送完成，发送多个字节数据的时候使用
TXIE	发送完成中断使能

接收器

如果将 USART_CR1 寄存器的 RE 位置 1，使能 USART 接收，使得接收器在 RX 线开始搜索起始位。在确定到起始位后就根据 RX 线电平状态把数据存放在接收移位寄存器内。接收完成后就把接收移位寄存器数据移到 RDR 内，并把 USART_SR 寄存器的 RXNE 位置 1，同时如果 USART_CR2 寄存器的 RXNEIE 置 1 的话可以产生中断。

在接收数据时，编程的时候有几个比较重要的标志位我们来总结下。

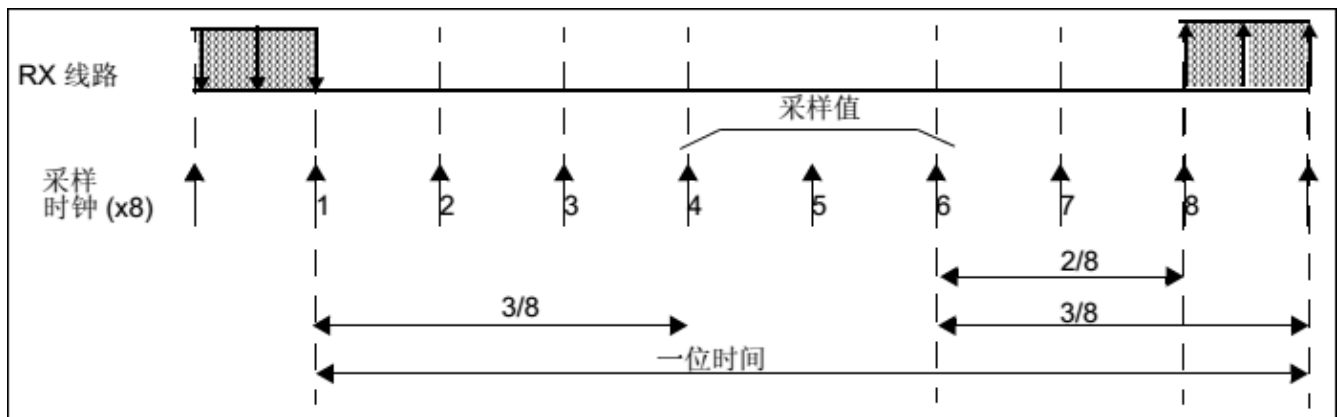
名称	描述
RE	接收使能
RXNE	读数据寄存器非空
RXNEIE	发送完成中断使能

为得到一个信号真实情况，需要用一個比这个信号频率高的采样信号去检测，称为过采样，这个采样信号的频率大小决定最后得到源信号准确度，一般频率越高得到的准确度越高，但为了得到越高频率采样信号越也困难，运算和功耗等等也会增加，所以一般选择合适就好。

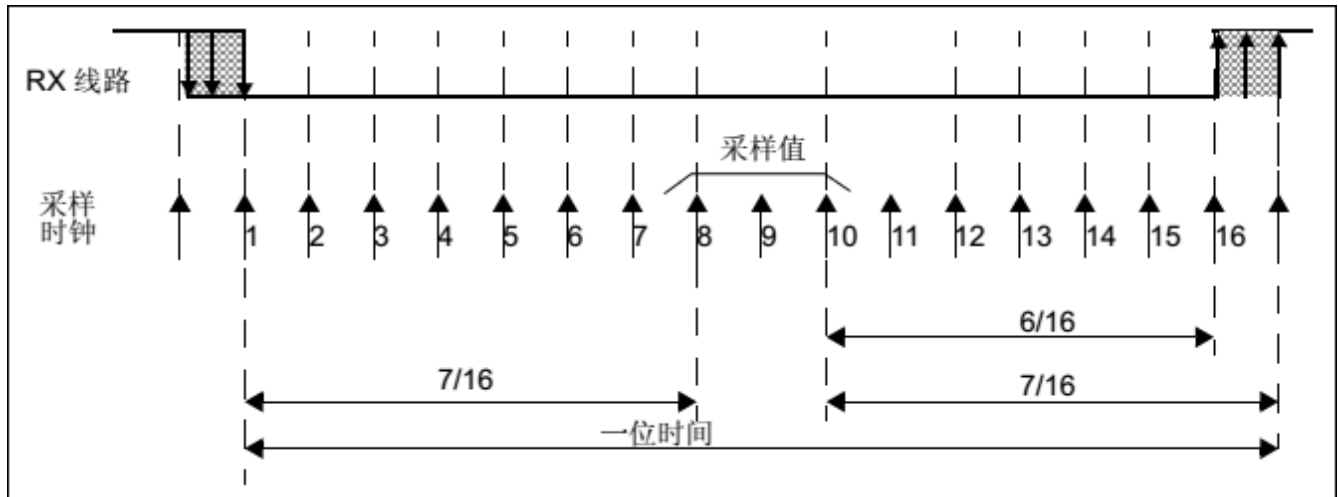
接收器可配置为不同过采样技术，以实现从噪声中提取有效的数据。USART_CR1 寄存器的 OVER8 位用来选择不同的采样方法，如果 OVER8 位设置为 1 采用 8 倍过采样，即用 8 个采样信号采样一位数据；如果 OVER8 位设置为 0 采用 16 倍过采样，即用 16 个采样信号采样一位数据。

USART 的起始位检测需要用到特定序列。如果在 RX 线识别到该特定序列就认为是检测到了起始位。起始位检测对使用 16 倍或 8 倍过采样的序列都是一样的。该特定序列为：1110X0X0X0000，其中 X 表示电平任意，1 或 0 皆可。

8 倍过采样速度更快，最高速度可达 $f_{PCLK}/8$ ， f_{PCLK} 为 USART 时钟，采样过程见图 20_19。使用第 4、5、6 次脉冲的值决定该位的电平状态。



16 倍过采样速度虽然没有 8 倍过采样那么快，但得到的数据更加精准，其最大速度为 $f_{PCLK}/16$ ，采样过程见图 20_20。使用第 8、9、10 次脉冲的值决定该位的电平状态。



19.3.4 □ 小数波特率生成

波特率指数据信号对载波的调制速率，它用单位时间内载波调制状态改变次数来表示，单位为波特。比特率指单位时间内传输的比特数，单位 bit/s(bps)。对于 USART 波特率与比特率相等，以后不区分这两个概念。波特率越大，传输速率越快。

USART 的发送器和接收器使用相同的波特率。计算公式如下：

$$\text{波特率} = \frac{f_{PLCK}}{8 \times (2 - \text{OVER8}) \times \text{USARTDIV}}$$

公式 20-1 波特率计算

其中，fPLCK 为 USART 时钟，参考表 20_3；OVER8 为 USART_CR1 寄存器的 OVER8 位对应的值，USARTDIV 是一个存放在波特率寄存器 (USART_BRR) 的一个无符号定点数。其中 DIV_Mantissa[11:0] 位定义 USARTDIV 的整数部分，DIV_Fraction[3:0] 位定义 USARTDIV 的小数部分，DIV_Fraction[3] 位只有在 OVER8 位为 0 时有效，否则必须清零。

例如，如果 OVER8=0，DIV_Mantissa=24 且 DIV_Fraction=10，此时 USART_BRR 值为 0x18A；那么 USARTDIV 的小数位 10/16=0.625；整数位 24，最终 USARTDIV 的值为 24.625。

如果 OVER8=0 并且知道 USARTDIV 值为 27.68，那么 DIV_Fraction=16*0.68=10.88，最接近的正整数为 11，所以 DIV_Fraction[3:0] 为 0xB；DIV_Mantissa= 整数 (27.68)=27，即位 0x1B。

如果 OVER8=1 情况类似，只是把计算用到的权值由 16 改为 8。

波特率的常用值有 2400、9600、19200、115200。下面以实例讲解如何设定寄存器值得到波特率的值。

由表 20_3 可知 USART1 和 USART6 使用 APB2 总线时钟，最高可达 84MHz，其他 USART 的最高频率为 42MHz。我们选取 USART1 作为实例讲解，即 fPLCK=84MHz。当我们使用 16 倍过采样时即 OVER8=0，为得到 115200bps 的波特率，此时：

$$115200 = \frac{84000000}{16 * USARTDIV}$$

解得 USARTDIV=45.57，可算得 DIV_Fraction=0x9 (0.57*24=9.12 取整，在 BRR 寄存器中，表示小数位的有 4 位)，DIV_Mantissa=0x2D，即应该设置 USART_BRR 的值为 0x2D9。

在计算 DIV_Fraction 时经常出现小数情况，经过我们取舍得到整数，这样会导致最终输出的波特率较目标值略有偏差。下面我们从 USART_BRR 的值为 0x2D9 开始计算得出实际输出的波特率大小。

由 USART_BRR 的值为 0x2D9，可得 DIV_Fraction=45，DIV_Mantissa=9，所以 USART-DIV=45+9/16=45.5625，所以实际波特率为：115226；这个值跟我们的目标波特率相差很小，这么小的误差在正常通信的允许范围内。

8 倍过采样时计算情况原理是一样的。

19.3.5 校验控制

STM32F4xx 系列控制器 USART 支持奇偶校验。当使用校验位时，串口传输的长度将是 8 位的数据帧加上 1 位的校验位总共 9 位，此时 USART_CR1 寄存器的 M 位需要设置为 1，即 9 数据位。将 USART_CR1 寄存器的 PCE 位置 1 就可以启动奇偶校验控制，奇偶校验由硬件自动完成。启动了奇偶校验控制之后，在发送数据帧时会自动添加校验位，接收数据时自动验证校验位。接收数据时如果出现奇偶校验位验证失败，会见 USART_SR 寄存器的 PE 位置 1，并可以产生奇偶校验中断。

使能了奇偶校验控制后，每个字符帧的格式将变成：起始位 + 数据帧 + 校验位 + 停止位。

19.3.6 中断控制

USART 有多个中断请求事件，具体见表 20-4。

表 20-4 USART 中断请求

中断事件	事件标志	使能控制位
发送数据寄存器为空	TXE	TXEIE
CTS 标志	CTS	CTSIE
发送完成	TC	TCIE
准备好读取接收到的数据	RXNE	RXNEIE
检测到上溢错误	ORE	RXNEIE
检测到空闲线路	IDLE	IDLEIE
奇偶校验错误	PE	PEIE
断路标志	LBD	LBDIE
多缓冲通信中的噪声标志、上溢错误和帧错误	NF/ORE/FE	EIE

19.4 USART 初始化结构体详解

HAL 库函数对每个外设都建立了一个初始化结构体，比如 USART_InitTypeDef，结构体成员用于设置外设工作参数，并由外设初始化配置函数，比如 USART_Init() 调用，这些设定参数将会设置外设相应的寄存器，达到配置外设工作环境的目的。

初始化结构体和初始化库函数配合使用是 HAL 库精髓所在，理解了初始化结构体每个成员意义基本上就可以对该外设运用自如了。初始化结构体定义在 stm32f4xx_hal_usart.h 文件中，初始化库函数定义在 stm32f4xx_hal_usart.c 文件中，编程时我们可以结合这两个文件内注释使用。

19.4.1 USART 初始化结构体

```
typedef struct {  
    uint32_t BaudRate;           //波特率  
    uint32_t WordLength;         //字长  
    uint32_t StopBits;           //停止位  
    uint32_t Parity;             //校验位  
    uint32_t Mode;               //UART 模式  
    uint32_t CLKPolarity;        //CLK极性  
    uint32_t CLKPhase;           //CLK相位  
    uint32_t CLKLastBit;         //最尾位时钟脉冲  
} USART_InitTypeDef;
```

1)BaudRate: 波特率设置。一般设置为 2400、9600、19200、115200。HAL 库函数会根据设定值计算得到 UARTDIV 值, 见前面公式, 并设置 UART_BRR 寄存器值。

2)WordLength: 数据帧字长, 可选 8 位或 9 位。它设定 UART_CR1 寄存器的 M 位的值。如果没有使能奇偶校验控制, 一般使用 8 数据位; 如果使能了奇偶校验则一般设置为 9 数据位。

3)StopBits: 停止位设置, 可选 0.5 个、1 个、1.5 个和 2 个停止位, 它设定 USART_CR2 寄存器的 STOP[1:0] 位的值, 一般我们选择 1 个停止位。

4)Parity: 奇偶校验控制选择, 可选 USART_PARITY_NONE (无校验)、USART_PARITY_EVEN (偶校验) 以及 USART_PARITY_ODD (奇校验), 它设定 UART_CR1 寄存器的 PCE 位和 PS 位的值。

5)Mode: UART 模式选择, 有 USART_MODE_RX 和 USART_MODE_TX, 允许使用逻辑或运算选择两个, 它设定 USART_CR1 寄存器的 RE 位和 TE 位。

6)USART_LastBit: 选择在发送最后一个数据位的时候时钟脉冲是否在 SCLK 引脚输出, 可以是不输出脉冲 (USART_LASTBIT_DISABLE)、输出脉冲 (USART_LASTBIT_ENABLE)。它设定 USART_CR2 寄存器的 LBCL 位的值。

19.5 USART1 接发通信实验

USART 只需两根信号线即可完成双向通信，对硬件要求低，使得很多模块都预留 USART 接口来实现与其他模块或者控制器进行数据传输，比如 GSM 模块、WIFI 模块、蓝牙模块等等。在硬件设计时，注意还需要一根“共地线”。

我们经常使用 USART 来实现控制器与电脑之间的数据传输。这使得我们调试程序非常方便，比如我们可以把一些变量的值、函数的返回值、寄存器标志位等等通过 USART 发送到串口调试助手，这样我们可以非常清楚程序的运行状态，当我们正式发布程序时再把这些调试信息去除即可。

我们不仅仅可以将数据发送到串口调试助手，我们还可以在串口调试助手发送数据给控制器，控制器程序根据接收到的数据进行下一步工作。

首先，我们来编写一个程序实现开发板与电脑通信，在开发板上电时通过 USART 发送一串字符串给电脑，然后开发板进入中断接收等待状态，如果电脑有发送数据过来，开发板就会产生中断，我们在中断服务函数接收数据，并马上把数据返回发送给电脑。

19.5.1 硬件设计

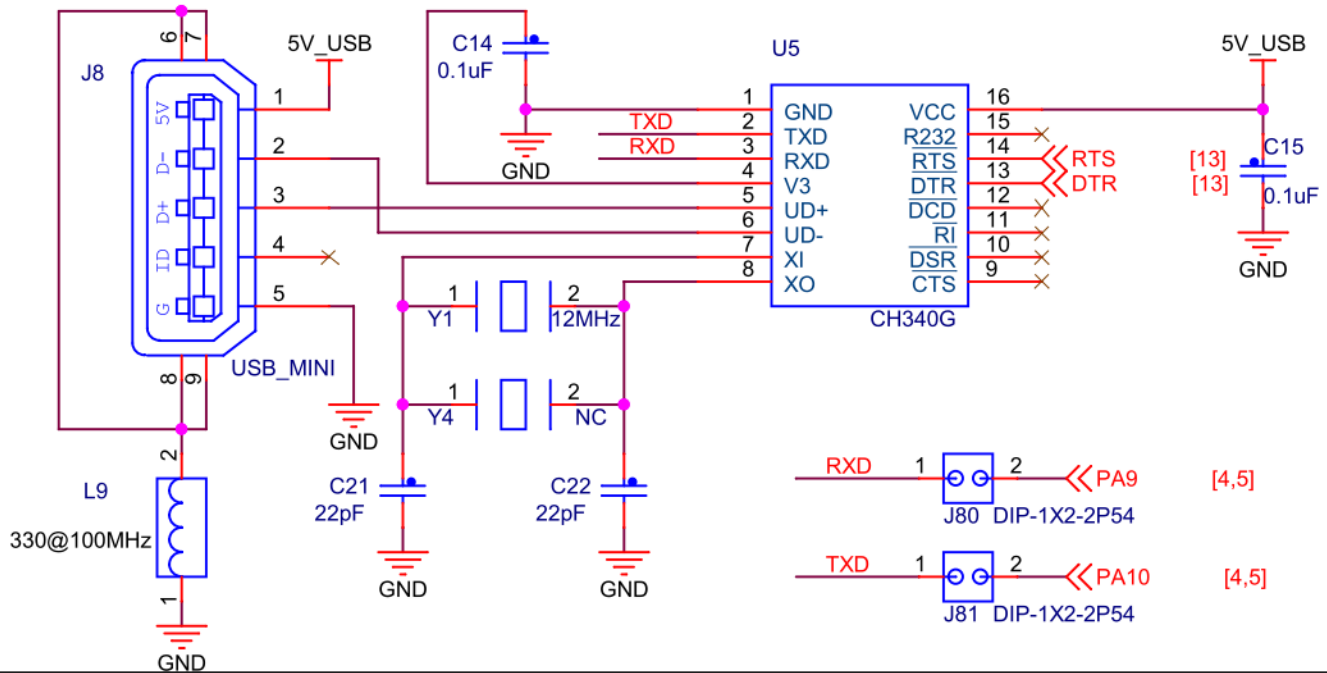
为利用 USART 实现开发板与电脑通信，需要用到一个 USB 转 USART 的 IC，我们选择 CH340G 芯片来实现这个功能，CH340G 是一个 USB 总线的转接芯片，实现 USB 转 USART、USB 转 IrDA 红外或者 USB 转打印机接口，我们使用其 USB 转 USART 功能。具体电路设计见图 20_9a、图 20_9b。

这里我们的 F407-霸天虎、F429-挑战者和 F407-骄阳电机开发板使用的都是 USART1，但是 F407-霸天虎、F429-挑战者这两者和 F407-骄阳上使用的 USART1 引脚是不同的：

- F407-霸天虎、F429-挑战者：使用 PB6 (USART1_TX)、PB7 (USART1_RX)
- F407-骄阳：使用 PA9 (USART1_TX)、PA10 (USART1_RX)

我们将 CH340G 的 TXD 引脚通过跳帽与 USART1 的 RX 引脚连接，CH340G 的 RXD 引脚也通过跳帽与 USART1 的 TX 引脚连接。CH340G 芯片集成在开发板上，其地线 (GND) 已与控制器的 GND 连通。如果我们想使用串口 2，那么就可以把开发板中的默认连接的跳帽拔掉，然后用

USB转串口



天猫: <https://yehuosm.tmall.com>

USB转串口

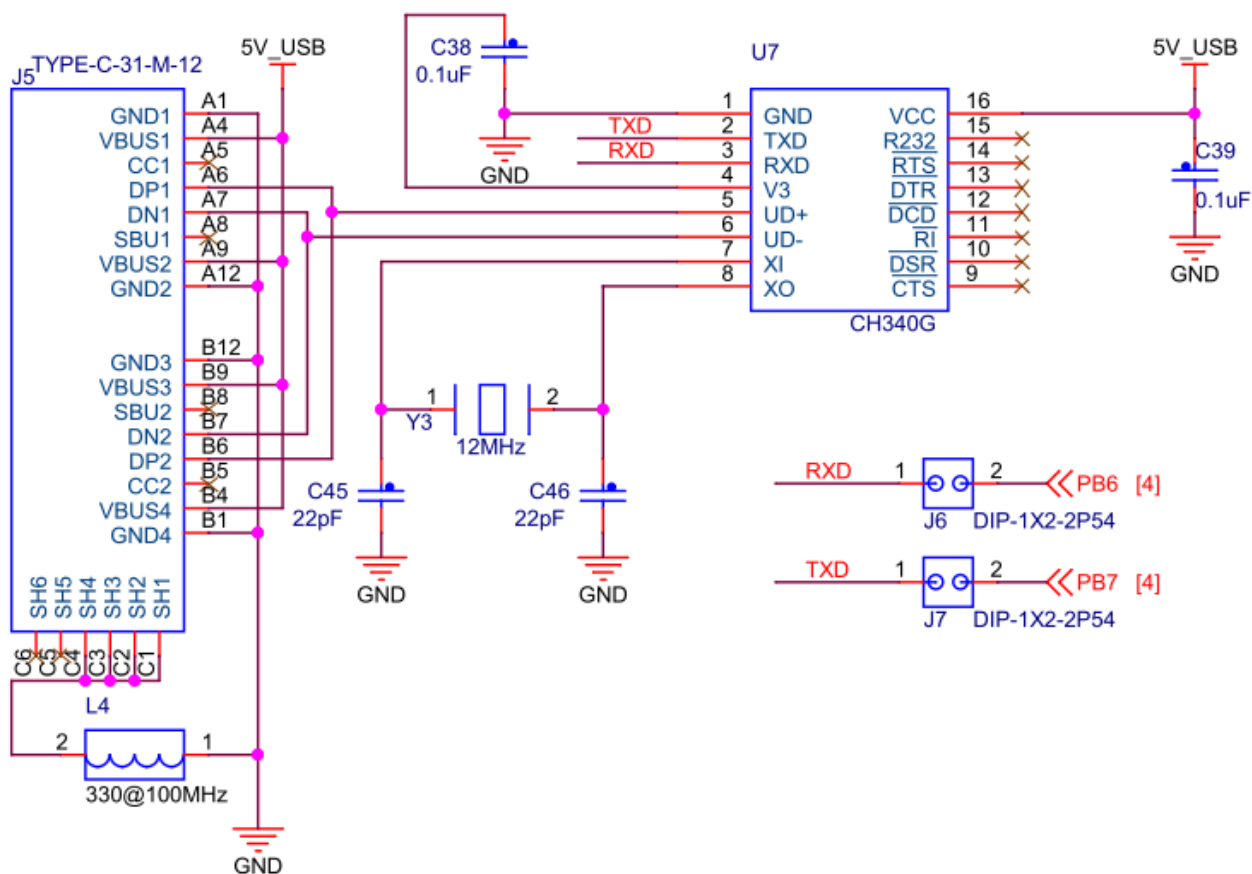


图 20-9b F407-骄阳电机开发板：USB 转串口硬件设计

19.5.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_debug_usart.c 和 bsp_debug_usart.h 文件用来存放 USART 驱动程序及相关宏定义。

19.5.2.1 编程要点

- 1) 使能 RX 和 TX 引脚 GPIO 时钟和 USART 时钟；
- 2) 初始化 GPIO，并将 GPIO 复用到 USART 上；
- 3) 配置 USART 参数；
- 4) 配置中断控制器并使能 USART 接收中断；
- 5) 使能 USART；
- 6) 在 USART 接收中断服务函数实现数据接收和发送。

19.5.2.2 代码分析

19.5.2.2.1 GPIO 和 USART 宏定义

列表 1: 代码清单 20-1a F407-霸天虎、F429-挑战者开发板：GPIO 和 USART 宏定义

```
//串口波特率

#define DEBUG_USART_BAUDRATE 115200

//引脚定义
/*****/
#define DEBUG_USART                USART1
#define DEBUG_USART_CLK_ENABLE()   __USART1_CLK_ENABLE();

#define RCC_PERIPHCLK_UARTx        RCC_PERIPHCLK_USART1
#define RCC_UARTxCLKSOURCE_SYSCLK  RCC_USART1CLKSOURCE_SYSCLK

#define DEBUG_USART_RX_GPIO_PORT   GPIOA
#define DEBUG_USART_RX_GPIO_CLK_ENABLE() __GPIOA_CLK_ENABLE()
```

(下页继续)

(续上页)

```
#define DEBUG_USART_RX_PIN          GPIO_PIN_10
#define DEBUG_USART_RX_AF          GPIO_AF7_USART1

#define DEBUG_USART_TX_GPIO_PORT    GPIOA
#define DEBUG_USART_TX_GPIO_CLK_ENABLE()  __GPIOA_CLK_ENABLE()
#define DEBUG_USART_TX_PIN          GPIO_PIN_9
#define DEBUG_USART_TX_AF          GPIO_AF7_USART1

#define DEBUG_USART_IRQHandler      USART1_IRQHandler
#define DEBUG_USART_IRQ             USART1_IRQn
/*****/
```

列表 2: 代码清单 20-1b F407-骄阳电机开发板: GPIO 和 USART 宏定义

```
//串口波特率
#define DEBUG_USART_BAUDRATE 115200

//引脚定义
/*****/
#define DEBUG_USART          USART1
#define DEBUG_USART_CLK_ENABLE()  __USART1_CLK_ENABLE();

#define RCC_PERIPHCLK_UARTx    RCC_PERIPHCLK_USART1
#define RCC_UARTxCLKSOURCE_SYSCLK  RCC_USART1CLKSOURCE_SYSCLK

#define DEBUG_USART_RX_GPIO_PORT    GPIOB
#define DEBUG_USART_RX_GPIO_CLK_ENABLE()  __GPIOB_CLK_ENABLE()
#define DEBUG_USART_RX_PIN          GPIO_PIN_7
#define DEBUG_USART_RX_AF          GPIO_AF7_USART1

#define DEBUG_USART_TX_GPIO_PORT    GPIOB
```

(下页继续)

(续上页)

```
#define DEBUG_USART_TX_GPIO_CLK_ENABLE()    __GPIOB_CLK_ENABLE()
#define DEBUG_USART_TX_PIN                  GPIO_PIN_6
#define DEBUG_USART_TX_AF                   GPIO_AF7_USART1

#define DEBUG_USART_IRQHandler              USART1_IRQHandler
#define DEBUG_USART_IRQ                     USART1_IRQn
/*****/
```

使用宏定义方便程序移植和升级，根据图 20_9a 或图 20_9a 电路，我们选择使用 USART1，设定波特率为 115200，一般我们会默认使用“8-N-1”参数，即 8 个数据位、不用奇偶校验、一位停止位。查阅表 20_3 可知 USART1 的 TX 线可对于 PA9 和 PB6 引脚，RX 线可对于 PA10 和 PB7 引脚，F407-霸天虎、F429-挑战者选择 PA9 以及 PA10 引脚，F407-骄阳选择 PB6 以及 PB7 引脚。最后我们定义中断相关参数并使能串口接收中断。

19.5.2.2.2 USART 初始化配置

列表 3: 代码清单 20-2 USART 初始化配置

```
/**
 * @brief  DEBUG_USART GPIO 配置，工作模式配置。115200 8-N-1
 * @param  无
 * @retval 无
 */
void DEBUG_USART_Config(void)
{
    UartHandle.Instance      = DEBUG_USART; //USART1 句柄

    UartHandle.Init.BaudRate  = DEBUG_USART_BAUDRATE; //波特率
    UartHandle.Init.WordLength = UART_WORDLENGTH_8B;  //8 位字长
    UartHandle.Init.StopBits  = UART_STOPBITS_1;      //一个停止位
```

(下页继续)

(续上页)

```
UartHandle.Init.Parity      = UART_PARITY_NONE;           //无奇偶校验
UartHandle.Init.HwFlowCtl   = UART_HWCONTROL_NONE;       //无硬件流控
UartHandle.Init.Mode        = UART_MODE_TX_RX;           //收发模式

HAL_UART_Init(&UartHandle);

/* 使能串口接收断 */
__HAL_UART_ENABLE_IT(&UartHandle, UART_IT_RXNE);
}
```

函数体中 `UartHandle` 是定义为 `UART_HandleTypeDef` 结构体类型的全局变量，它管理着串口的所有配置。该例程中 `DEBUG_USART_Config` 函数初始化 USART 与 MCU 硬件无关的东西：例如串口协议，其中包括波特率，奇偶校验，停止位等等，这些设置和使用什么样的 MCU 没有任何关系，可以使用 F1 的 MCU，也可以是 F2…F4，甚至是 PC 上的串口。所以就把串口抽象成为一个“串口”。至于对具体 mcu 底层硬件相关的配置如引脚、时钟、DMA、中断等是在 `HAL_UART_MspInit(UART_HandleTypeDef *huart)` 函数中完成的，该函数被 `HAL_UART_Init` 函数所调用。

需要特别指出的是在 `HAL_UART_Init` 调用 `HAL_UART_MspInit` 库函数的函数原型是：

列表 4: 代码清单 20-3 HAL_UART_MspInit 库函数定义

```
__weak void HAL_UART_MspDeInit(UART_HandleTypeDef *huart)

{

    /* 防止未使用的参数编译警告 */

    UNUSED(huart);

}
```

`__weak` 表示弱定义，表示如果你自己定义了同名的函数就不用他，如果你没定义就使用这个弱

函数。其中 UNUSED 函数只是为了防止未使用的参数编译警告，其实质是什么也不做。下面列举的是 HAL_UART_MspInit 强函数的定义，它是实际被 HAL_UART_Init 函数所调用的函数。

列表 5: 代码清单 20-4 HAL_UART_MspInit 用户强函数定义

```
/*  
  
* @brief UART MSP 初始化  
  
* @param huart: UART handle  
  
* @retval 无  
  
*/  
  
void HAL_UART_MspInit (UART_HandleTypeDef *huart)  
{  
  
    GPIO_InitTypeDef GPIO_InitStruct;  
  
    DEBUG_USART_CLK_ENABLE();  
  
    DEBUG_USART_RX_GPIO_CLK_ENABLE();  
    DEBUG_USART_TX_GPIO_CLK_ENABLE();  
  
    /* USART1 GPIO Configuration  
    //对于 F407-霸天虎开发板、F429-挑战者开发板:  
        PA9 -----> USART1_TX  
        PA10 -----> USART1_RX  
    //对于 F407-骄阳电机开发板:  
        PB6 -----> USART1_TX
```

(下页继续)

(续上页)

```
        PB7      -----> USART1_RX
    */

    /* 配置 Tx 引脚为复用功能 */
    GPIO_InitStruct.Pin = DEBUG_USART_TX_PIN;
    GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;           //复用推挽输出
    GPIO_InitStruct.Pull = GPIO_PULLUP;              //上拉
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH; //高速
    GPIO_InitStruct.Alternate = DEBUG_USART_TX_AF;    //复用为 USART1
    HAL_GPIO_Init(DEBUG_USART_TX_GPIO_PORT, &GPIO_InitStruct);

    /* 配置 Rx 引脚为复用功能 */
    GPIO_InitStruct.Pin = DEBUG_USART_RX_PIN;
    GPIO_InitStruct.Alternate = DEBUG_USART_RX_AF;
    HAL_GPIO_Init(DEBUG_USART_RX_GPIO_PORT, &GPIO_InitStruct);

    /* 抢占优先级 0, 子优先级 1*/
    HAL_NVIC_SetPriority(DEBUG_USART_IRQ, 0, 1); //抢占优先级 0, 子优先级 1

    HAL_NVIC_EnableIRQ(DEBUG_USART_IRQ);        //使能 USART1 中断通道

}
```

在 HAL 库中 IO 口初始化参数设置和复用映射配置是在函数 HAL_GPIO_Init 中一次性完成的。需要注意，我们要复用 PA9 和 PA10 或者 PB6 和 PB7 为串口发送接收相关引脚，我们需要配置 IO 口为复用，同时复用映射到串口 1。对于中断优先级配置，方法就非常简单只需要调用函数 HAL_NVIC_SetPriority 中断优先级配置和函数 HAL_NVIC_EnableIRQ 中断使能即可。

19.5.2.2.3 字符发送

列表 6: 代码清单 20-5 字符发送函数

```
/****** 发送字符串 *\******/  
  
void Usart_SendString(uint8_t *str)  
{  
  
    unsigned int k=0;  
  
    do {  
  
        HAL_UART_Transmit( &UartHandle, (uint8_t *) (str + k) ,1,1000);  
  
        k++;  
  
    } while (*(str + k) != '\0');  
  
}
```

Usart_SendString 函数用来发送一个字符串，它实际是调用 HAL_UART_Transmit 函数（这是一个阻塞的发送函数，无需重复判断串口是否发送完成）发送每个字符，直到遇到空字符才停止发送。最后使用循环检测发送完成的事件标志来实现保证数据发送完成后才退出函数。

19.5.2.2.4 USART 中断服务函数

列表 7: 代码清单 20-6 USART 中断服务函数

```
void DEBUG_USART_IRQHandler(void)

{

    uint8_t ch;

    if ( __HAL_UART_GET_FLAG( &UartHandle, UART_FLAG_RXNE ) != RESET)
    {

        ch= ( uint16_t) READ_REG (UartHandle.Instance->DR);

        WRITE_REG ( UartHandle.Instance->DR,ch);

    }

}
```

这段代码是存放在 stm32f4xx_it.c 文件中的，该文件用来集中存放外设中断服务函数。当我们使能了中断并且中断发生时就会执行中断服务函数。

我们在代码清单 20_2 使能了 USART 接收中断，当 USART 有接收到数据就会执行 DEBUG_USART_IRQHandler 函数。__HAL_UART_GET_FLAG 函数用来获取中断事件标志。使用 if 语句来判断是否是真的产生 USART 数据接收这个中断事件，如果是真的就使用 USART 数据读取函数 READ_REG 读取数据赋值给 ch，读取过程会软件清除 UART_FLAG_RXNE 标志位。最后再调用 USART 写函数 WRITE_REG 把数据又发送给源设备。

19.5.2.2.5 主函数

列表 8: 代码清单 20-7 主函数

```
int main(void)

{

    HAL_Init();

    /* 配置系统时钟 168MHz */

    SystemClock_Config();

    /* 初始化 USART 配置模式为 115200 8-N-1, 中断接收 */

    DEBUG_USART_Config();

    /* 调用 printf 函数, 因为重定向了 fputc, printf 的内容会输出到串口 */

    printf("欢迎使用野火开发板\n");

    /* 自定义函数方式 */

    Usart_SendString( (uint8_t *)

        "自定义函数输出: 这是一个串口中断 接收回显实验 n" );

    while (1) {

    }

}
```

首先我们需要调用 `SystemClock_Config` 函数配置系统时钟，调用 `Debug_USART_Config` 函数完成 USART 初始化配置，包括 GPIO 配置，USART 配置，接收中断使用等等信息。数据的回传在中断函数实现。

19.5.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板，此时串口调试助手即可收到开发板发过来的数据。我们在串口调试助手发送区域输入任意字符，点击发送按钮，马上在串口调试助手接收区即可看到相同的字符。



图 20-10 实验现象

19.6 USART1 指令控制 RGB 彩灯实验

在学习 C 语言时我们经常使用 C 语言标准函数库输入输出函数，比如 `printf`、`scanf`、`getchar` 等等。为让开发板也支持这些函数需要把 USART 发送和接收函数添加到这些函数的内部函数内。

正如之前所讲，可以在串口调试助手输入指令，让开发板根据这些指令执行一些任务，现在我们编写程序让开发板接收 USART 数据，然后根据数据内容控制 RGB 彩灯的颜色。

由于 F407-骄阳电机开发板上面没有 RGB 彩灯，所以本实验仅适用于 F407-霸天虎、F429-挑战者开发板。但是，只要稍微修改一下代码也是可以控制 F407-骄阳上的 LED 灯的，具体代码可以去看相应的例程，这里不再列出。

19.6.1 硬件设计

硬件设计同第一个实验。LED 灯的接线引脚可参考前面点亮 LED 灯的章节。

19.6.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_usart.c 和 bsp_usart.h 文件用来存放 USART 驱动程序及相关宏定义。

19.6.2.1 编程要点

- 1) 初始化配置 RGB 彩色灯 GPIO；
- 2) 使能 RX 和 TX 引脚 GPIO 时钟和 USART 时钟；
- 3) 初始化 GPIO，并将 GPIO 复用到 USART 上；
- 4) 配置 USART 参数；
- 5) 使能 USART；
- 6) 获取指令输入，根据指令控制 RGB 彩色灯。

19.6.2.2 代码分析

19.6.2.2.1 GPIO 和 USART 宏定义

列表 9: 代码清单 20-8a F407-霸天虎、F429-挑战者开发板: GPIO 和 USART 宏定义

```
//串口波特率

#define DEBUG_USART_BAUDRATE 115200

//引脚定义
/*****/

#define DEBUG_USART                USART1
#define DEBUG_USART_CLK_ENABLE()   __USART1_CLK_ENABLE();

#define RCC_PERIPHCLK_UARTx        RCC_PERIPHCLK_USART1
#define RCC_UARTxCLKSOURCE_SYSCLK  RCC_USART1CLKSOURCE_SYSCLK

#define DEBUG_USART_RX_GPIO_PORT   GPIOA
#define DEBUG_USART_RX_GPIO_CLK_ENABLE() __GPIOA_CLK_ENABLE()
#define DEBUG_USART_RX_PIN         GPIO_PIN_10
#define DEBUG_USART_RX_AF          GPIO_AF7_USART1

#define DEBUG_USART_TX_GPIO_PORT   GPIOA
#define DEBUG_USART_TX_GPIO_CLK_ENABLE() __GPIOA_CLK_ENABLE()
#define DEBUG_USART_TX_PIN         GPIO_PIN_9
#define DEBUG_USART_TX_AF          GPIO_AF7_USART1

#define DEBUG_USART_IRQHandler     USART1_IRQHandler
#define DEBUG_USART_IRQ            USART1_IRQn
/*****/
```

列表 10: 代码清单 20-8b F407-骄阳电机开发板: GPIO 和 USART 宏定义

```
//串口波特率
#define DEBUG_USART_BAUDRATE 115200

//引脚定义
/*****/

#define DEBUG_USART                                USART1
#define DEBUG_USART_CLK_ENABLE()                  __USART1_CLK_ENABLE();

#define RCC_PERIPHCLK_UARTx                        RCC_PERIPHCLK_USART1
#define RCC_UARTxCLKSOURCE_SYSCLK                  RCC_USART1CLKSOURCE_SYSCLK

#define DEBUG_USART_RX_GPIO_PORT                   GPIOB
#define DEBUG_USART_RX_GPIO_CLK_ENABLE()           __GPIOB_CLK_ENABLE()
#define DEBUG_USART_RX_PIN                          GPIO_PIN_7
#define DEBUG_USART_RX_AF                          GPIO_AF7_USART1

#define DEBUG_USART_TX_GPIO_PORT                   GPIOB
#define DEBUG_USART_TX_GPIO_CLK_ENABLE()           __GPIOB_CLK_ENABLE()
#define DEBUG_USART_TX_PIN                          GPIO_PIN_6
#define DEBUG_USART_TX_AF                          GPIO_AF7_USART1

#define DEBUG_USART_IRQHandler                     USART1_IRQHandler
#define DEBUG_USART_IRQ                            USART1_IRQn
/*****/
```

使用宏定义方便程序移植和升级，这里我们可以 USART1，设定波特率为 115200。

19.6.2.2.2 USART 初始化配置

列表 11: 代码清单 20-9 USART 初始化配置

```
void UARTx_Config(void)

{

    GPIO_InitTypeDef GPIO_InitStructure;

    DEBUG_USART_RX_GPIO_CLK_ENABLE();
    DEBUG_USART_TX_GPIO_CLK_ENABLE();

    /* 使能 UART 时钟 */
    DEBUG_USART_CLK_ENABLE();

    /* USART1 GPIO Configuration
    //对于 F407-霸天虎开发板、F429-挑战者开发板:
        PA9 -----> USART1_TX
        PA10 -----> USART1_RX
    //对于 F407-骄阳电机开发板:
        PB6 -----> USART1_TX
        PB7 -----> USART1_RX
    */

    /* 配置 Tx 引脚为复用功能 */
    GPIO_InitStructure.Pin = DEBUG_USART_TX_PIN;
    GPIO_InitStructure.Mode = GPIO_MODE_AF_PP; //复用推挽输出
    GPIO_InitStructure.Pull = GPIO_PULLUP; //上拉
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH; //高速
    GPIO_InitStructure.Alternate = DEBUG_USART_TX_AF; //复用为 USART1
    HAL_GPIO_Init(DEBUG_USART_TX_GPIO_PORT, &GPIO_InitStructure);
```

(下页继续)

(续上页)

```
/* 配置 Rx 引脚为复用功能 */
GPIO_InitStruct.Pin = DEBUG_USART_RX_PIN;
GPIO_InitStruct.Alternate = DEBUG_USART_RX_AF;
HAL_GPIO_Init(DEBUG_USART_RX_GPIO_PORT, &GPIO_InitStruct);

/* 配置串口工作模式: 115200 8-N-1 */
UartHandle.Instance = DEBUG_USART; //USART1 句柄
UartHandle.Init.BaudRate = DEBUG_USART_BAUDRATE; //波特率
UartHandle.Init.WordLength = UART_WORDLENGTH_8B; //8 位字长
UartHandle.Init.StopBits = UART_STOPBITS_1; //一个停止位
UartHandle.Init.Parity = UART_PARITY_NONE; //无奇偶校验
UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE; //无硬件流控
UartHandle.Init.Mode = UART_MODE_TX_RX; //收发模式
HAL_UART_Init(&UartHandle);
}
```

在这个函数中我们并没有使用 `HAL_UART_MspInit` 函数，为了直观与简洁，而恶是直接在 `UARTx_Config` 函数中进行引脚、时钟等配置，在以后的部分例程我们也会大量使用这种方式。

使用 `GPIO_InitTypeDef` 和 `USART_InitTypeDef` 结构体定义一个 GPIO 初始化变量以及一个 USART 初始化变量，这两个结构体内容我们之前已经有详细讲解。

调用 `UARTx_RX_GPIO_CLK_ENABLE` 和 `UARTx_TX_GPIO_CLK_ENABLE` 函数开启 GPIO 端口时钟，使用 GPIO 之前必须开启对应端口的时钟。

初始化配置 RX 线和 TX 线引脚为复用功能，并将指定的 GPIO 连接至 USART1，然后配置串口的工作参数为 115200-8-N-1。最后调用 `HAL_UART_Init` 函数初始化 USART。

19.6.2.2.3 重定向 printf 和 scanf 函数

列表 12: 代码清单 20-10 重定向输入输出函数

```
//重定向 c 库函数 printf 到串口 USARTx, 重定向后可使用 printf 函数

int fputc(int ch, FILE *f)

{

    /* 发送一个字节数据到串口 USARTx */

    HAL_UART_Transmit(&UartHandle, (uint8_t *)&ch, 1, 0xFFFF);

    return (ch);

}

//重定向 c 库函数 scanf 到串口 USARTx, 重写后可使用 scanf、getchar 等函数

int fgetc(FILE *f)

{

    int ch;

    /* 等待串口输入数据 */

    while (__HAL_UART_GET_FLAG(&UartHandle, UART_FLAG_RXNE) == RESET);

    HAL_UART_Receive(&UartHandle, (uint8_t *)&ch, 1, 0xFFFF);

    return (ch);

}
```

(下页继续)

(续上页)

```
}
```

在 C 语言 HAL 库中，fputc 函数是 printf 函数内部的一个函数，功能是将字符 ch 写入到文件指针 f 所指向文件的当前写指针位置，简单理解就是把字符写入到特定文件中。我们使用 USART 函数重新修改 fputc 函数内容，达到类似“写入”的功能。

fgetc 函数与 fputc 函数非常相似，实现字符读取功能。在使用 scanf 函数时需要注意字符输入格式。

还有一点需要注意的，使用 fput 和 fgetc 函数达到重定向 C 语言 HAL 库输入输出函数必须在 MDK 的工程选项把“Use MicroLIB”勾选上，Micorolib 是缺省 C 库的备选库，它对标准 C 库进行了高度优化使代码更少，占用更少资源。

为使用 printf、scanf 函数需要在文件中包含 stdio.h 头文件。

19.6.2.2.4 输出提示信息

列表 13: 代码清单 20-11 输出提示信息

```
static void Show_Message(void)
{
    printf("\r\n 这是一个通过串口通信指令控制 RGB 彩灯实验 \n");

    printf("使用 USART 参数为: %d 8-N-1 \n", USART_BAUDRATE);

    printf("开发板接到指令后控制 RGB 彩灯颜色, 指令对应如下: \n");

    printf(" 指令 ----- 彩灯颜色 \n");
```

(下页继续)

(续上页)

```
printf(" 1 ----- 红 \n");

printf(" 2 ----- 绿 \n");

printf(" 3 ----- 蓝 \n");

printf(" 4 ----- 黄 \n");

printf(" 5 ----- 紫 \n");

printf(" 6 ----- 青 \n");

printf(" 7 ----- 白 \n");

printf(" 8 ----- 灭 \n");

}
```

Show_Message 函数全部是调用 printf 函数，“打印”实验操作信息到串口调试助手。

19.6.2.2.5 主函数

列表 14: 代码清单 20-12 主函数

```
int main(void)
{
    char ch;
    /* 配置系统时钟为 168 MHz */
    SystemClock_Config();

    /* 初始化 RGB 彩灯 */
}
```

(下页继续)

(续上页)

```
LED_GPIO_Config();

/* 初始化 USART1 配置模式为 115200 8-N-1 */
UARTx_Config();

/* 打印指令输入提示信息 */
Show_Message();
while (1) {
    /* 获取字符指令 */
    ch=getchar();
    printf("接收到字符: %c\n",ch);

    /* 根据字符指令控制 RGB 彩灯颜色 */
    switch (ch) {
        case '1':
            LED_RED;
            break;
        case '2':
            LED_GREEN;
            break;
        case '3':
            LED_BLUE;
            break;
        case '4':
            LED_YELLOW;
            break;
        case '5':
            LED_PURPLE;
            break;
        case '6':
            LED_CYAN;
            break;
```

(下页继续)

(续上页)

```
        case '7':
            LED_WHITE;
            break;
        case '8':
            LED_RGBOFF;
            break;
        default:
            /* 如果不是指定指令字符，打印提示信息 */
            Show_Message();
            break;
    }
}
```

首先我们定义一个字符变量来存放接收到的字符。

接下来调用 `SystemClock_Config` 函数初始化系统时钟，调用 `LED_GPIO_Config` 函数完成 RGB 彩色 GPIO 初始化配置，该函数定义在 `bsp_led.c` 文件内。

调用 `USARTx_Config` 函数完成 USART 初始化配置。

`Show_Message` 函数使用 `printf` 函数打印实验指令说明信息。

`getchar` 函数用于等待获取一个字符，并返回字符。我们使用 `ch` 变量保持返回的字符，接下来判断 `ch` 内容执行对应的程序了。

我们使用 `switch` 语句判断 `ch` 变量内容，并执行对应的功能程序。

19.6.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板，此时串口调试助手即可收到开发板发过来的数据。我们在串口调试助手发送区域输入一个特定字符，点击发送按钮，RGB 彩色灯状态随之改变。